

CMM705 - Big Data Programming Coursework - Report

Name: Siyambaladuwa Pathiranage Dinith Heshan

IIT ID: 20250103 | **RGU ID:** 2515374

Module: CMM705 - Big Data Programming

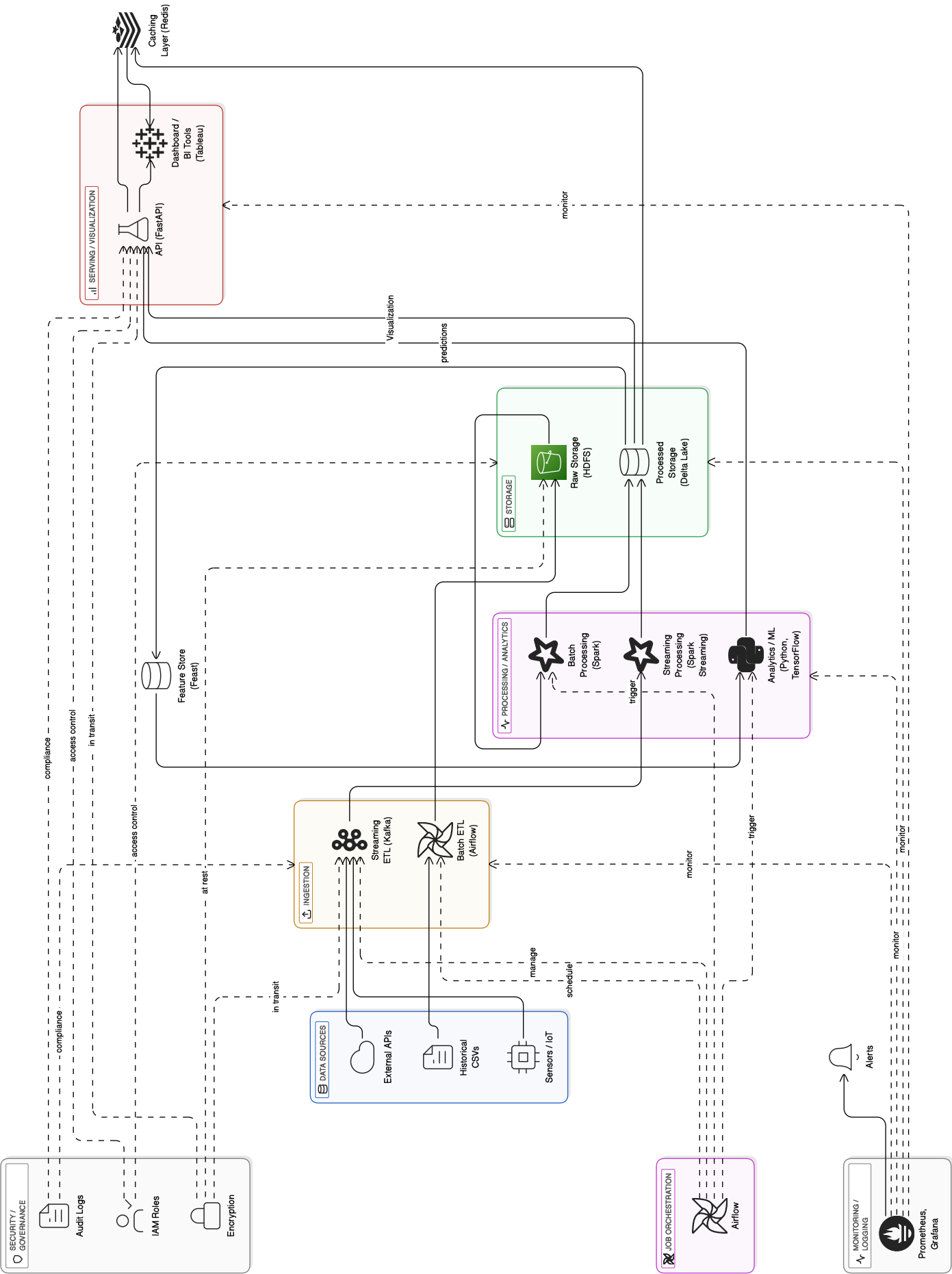
- Task 1 -

Section 01

The reference code was used to design the system architecture diagram using [Eraser.io](#). The complete set of diagram artifacts—including the **Eraser code**, **SVG**, and **PDF** exports—are provided under the following directory for reference.

- **bigdata-weather-analysis-sri-lanka/task-1/**

The **system architecture diagram** is presented on the **next page**.



Reference Code

```
title Sri Lanka Weather Analytics Platform - System Architecture

// 1. Data Sources (Blue)
Data Sources [color: blue, icon: database] {
  Historical CSVs [icon: file-text]
  Weather Sensors [icon: cpu, label: "Sensors / IoT"]
  External APIs [icon: cloud, label: "External APIs"]
}

// 2. Ingestion Layer (Orange)
Ingestion [color: orange, icon: upload] {
  Batch ETL [icon: airflow, label: "Batch ETL (Airflow)"]
  Streaming ETL [icon: kafka, label: "Streaming ETL (Kafka)"]
}

// 3. Storage Layer (Green)
Storage [color: green, icon: server] {
  Raw Data Storage [icon: aws-s3, label: "Raw Storage (HDFS)"]
  Processed Data Storage [icon: database, label: "Processed Storage (Delta Lake)"]
}

// 4. Processing / Analytics Layer (Purple)
Processing Analytics [color: purple, icon: activity, label: "Processing / Analytics"] {
  Batch Processing [icon: spark, label: "Batch Processing (Spark)"]
  Streaming Processing [icon: spark, label: "Streaming Processing (Spark Streaming)"]
  Analytics ML [icon: python, label: "Analytics / ML (Python, TensorFlow)"]
}

// 5. Serving / Visualization Layer (Red)
Serving Visualization [color: red, icon: bar-chart, label: "Serving / Visualization"] {
  API Layer [icon: flask, label: "API (FastAPI)"]
  Dashboard BI [icon: tableau, label: "Dashboard / BI Tools (Tableau)"]
}

// 6. Orchestration (Purple)
Orchestration [color: purple, icon: airflow, label: "Job Orchestration"] {
  Job Orchestration [icon: airflow, label: "Airflow"]
}

// 7. Monitoring / Logging (Gray)
Monitoring Logging [color: gray, icon: activity, label: "Monitoring / Logging"] {
  Observability [icon: prometheus, label: "Prometheus,Grafana"]
}

// 8. Security / Governance (Gray)
```

```
Security Governance [color: gray, icon: shield, label: "Security / Governance"] {
  IAM Roles [icon: user-check]
  Encryption [icon: lock]
  Audit Logs [icon: file-text, label: "Audit Logs"]
}

// Extras
Feature Store [icon: database, label: "Feature Store (Feast)"]
Caching Layer [icon: redis, label: "Caching Layer (Redis)"]
Alerts [icon: bell]

// Connections

// Batch Flow
Historical CSVs > Batch ETL
Batch ETL > Raw Data Storage
Raw Data Storage > Batch Processing
Batch Processing > Processed Data Storage

// Streaming Flow
Weather Sensors > Streaming ETL
External APIs > Streaming ETL
Streaming ETL > Streaming Processing
Streaming Processing > Processed Data Storage

// ML / Predictive Flow
Processed Data Storage > Feature Store
Feature Store > Analytics ML
Analytics ML > API Layer: predictions
API Layer > Dashboard BI
Processed Data Storage > Caching Layer
API Layer > Caching Layer
Dashboard BI < Caching Layer

// Orchestration
Job Orchestration --> Batch ETL: schedule
Job Orchestration --> Batch Processing: trigger
Job Orchestration --> Streaming ETL: manage
Job Orchestration --> Analytics ML: trigger

// Monitoring / Logging
Observability --> Ingestion: monitor
Observability --> Storage: monitor
Observability --> Processing Analytics: monitor
Observability --> Serving Visualization: monitor
Observability > Alerts
```

```
// Security / Governance
IAM Roles --> API Layer: access control
IAM Roles --> Storage: access control [color: Black]
Encryption --> Raw Data Storage: at rest
Encryption --> Streaming ETL: in transit
Encryption --> API Layer: in transit
Audit Logs --> API Layer: compliance
Audit Logs --> Ingestion: compliance

// Visualization
Processed Data Storage > API Layer: Visualization
```

Section 02

Role of Each Component and Overall Architecture Operation

The proposed **Sri Lanka Weather Analytics Platform** is designed to support **real-time ingestion, batch processing, interactive analytics, and predictive modeling**, while ensuring **scalability, availability, security, and minimal data duplication**. The system follows a **layered big data architecture**, where each component has a well-defined responsibility and interacts seamlessly with others.

1. Data Sources Layer

Role:

This layer represents all sources from which weather data originates.

- **Historical CSV Files**
Contain archived weather observations and location metadata. These files are ingested in batch mode to initialize and backfill the data lake.
- **Weather Sensors / IoT Devices**
Simulate real-time data generation such as temperature, rainfall, and wind speed collected continuously from districts.
- **External Weather APIs**
Provide supplementary real-time meteorological data from third-party services to enrich sensor data.

Purpose:

Supports both **batch and streaming data**, enabling historical trend analysis as well as near-real-time monitoring.

2. Ingestion Layer

Role:

Responsible for reliably collecting data from multiple sources and preventing duplication.

- **Batch ETL (Apache Airflow)**
Handles scheduled ingestion of historical CSV files. Airflow ensures idempotent batch jobs, retries on failure, and structured workflows.
- **Streaming ETL (Apache Kafka)**
Acts as a distributed message broker for real-time data streams from sensors and APIs. Kafka decouples producers and consumers, ensuring high throughput and fault tolerance.

Purpose:

Efficiently ingests large-scale data in both **batch and real-time modes** while maintaining data consistency.

3. Storage Layer

Role:

Provides scalable and durable storage for different stages of data.

- **Raw Data Storage (HDFS)**
Stores ingested data in its original format. This immutable layer supports reprocessing, auditing, and recovery.
- **Processed Data Storage (Delta Lake)**
Stores cleaned, transformed, and aggregated datasets with ACID guarantees, schema evolution, and versioning.

Purpose:

Separates raw and processed data to **avoid duplication**, support **reprocessing**, and enable **efficient analytics**.

4. Processing & Analytics Layer

Role:

Transforms raw data into meaningful insights.

- **Batch Processing (Apache Spark)**
Performs large-scale historical analysis such as long-term climate trends, district-wise summaries, and seasonal aggregations.
- **Streaming Processing (Spark Structured Streaming)**
Processes real-time data streams for rolling aggregates, anomaly detection, and near-real-time metrics.
- **Analytics & Machine Learning (Python, TensorFlow)**
Builds predictive models (e.g., rainfall forecasting, extreme weather prediction) using curated datasets and engineered features.

Purpose:

Supports **descriptive, diagnostic, and predictive analytics** across historical and live data.

5. Feature Store

Role:

The **Feature Store (Feast)** maintains reusable, versioned features derived from processed data.

- Ensures feature consistency between training and inference.
- Reduces redundant feature engineering efforts.

Purpose:

Enables **reliable and scalable ML workflows** for predictive analytics.

6. Serving & Visualization Layer

Role:

Delivers processed data and insights to end users.

- **API Layer (FastAPI)**
Exposes RESTful endpoints for dashboards and external consumers to access aggregated metrics and ML predictions.
- **Dashboard / BI Tools (Tableau)**
Provide interactive visualizations such as rainfall distribution, temperature trends, and district-level comparisons.
- **Caching Layer (Redis)**
Stores frequently accessed query results to reduce latency and backend load.

Purpose:

Ensures **low-latency, interactive analytics** and smooth user experience.

7. Orchestration Layer

Role:

- **Apache Airflow** orchestrates batch ETL, Spark jobs, and ML pipelines.
- Manages scheduling, dependencies, retries, and failure recovery.

Purpose:

Improves **reliability, automation, and operational efficiency** of the platform.

8. Monitoring, Logging, and Alerts

Role:

- **Prometheus** collects system and application metrics.
- **Grafana** visualizes performance dashboards.
- **Alerts** notify stakeholders of failures, latency spikes, or data ingestion issues.

Purpose:

Ensures **high availability, observability, and proactive issue detection**.

9. Security & Governance Layer

Role:

- **IAM Roles** enforce role-based access control across storage and APIs.

- **Encryption** secures data both at rest and in transit.
- **Audit Logs** maintain compliance and traceability of system actions.

Purpose:

Guarantees **data confidentiality, integrity, and regulatory compliance.**

Overall Architecture Operation

1. Historical data is ingested via batch pipelines, while real-time data flows through Kafka streams.
2. All raw data is stored in scalable storage before being processed by Spark.
3. Processed and aggregated data is stored in Delta Lake and served efficiently via APIs.
4. Machine learning models use curated features from the Feature Store to generate predictions.
5. Dashboards and applications consume data through cached APIs for low-latency access.
6. Airflow orchestrates workflows, while monitoring and security layers ensure system reliability and governance.

Justification of Technology Selection

Apache Airflow (Batch ETL & Orchestration)

Apache Airflow was selected to manage batch ingestion and workflow orchestration due to its strong support for **scheduling, dependency management, and fault tolerance**. It enables repeatable and idempotent execution of ETL pipelines, which is essential for ingesting historical weather datasets and coordinating Spark and machine learning workflows. Airflow's DAG-based design improves pipeline transparency and operational reliability.

Apache Kafka (Streaming ETL)

Apache Kafka was chosen as the real-time ingestion layer because of its **high throughput, low latency, and fault-tolerant message delivery**. Kafka efficiently decouples data producers (sensors and APIs) from downstream consumers, allowing the system to scale horizontally while supporting continuous real-time weather data streams.

HDFS (Raw Data Storage)

HDFS was selected for raw data storage due to its ability to **store large volumes of structured and semi-structured data reliably and cost-effectively**. Its distributed architecture supports high availability and data replication, making it suitable for long-term archival of immutable weather data while enabling reprocessing and auditing.

Delta Lake (Processed Data Storage)

Delta Lake was chosen for processed data storage because it provides **ACID transaction guarantees, schema enforcement, and versioning** on top of data lake storage. These features are critical for maintaining data consistency, preventing duplication, and supporting incremental updates required for analytical workloads.

Apache Spark (Batch & Streaming Processing)

Apache Spark was selected for both batch and streaming analytics due to its **in-memory distributed processing capabilities**. Spark efficiently handles large-scale historical climate analysis while also supporting near-real-time processing through Structured Streaming. This unified engine reduces system complexity and improves performance.

Python & TensorFlow (Analytics and Machine Learning)

Python was chosen for analytics due to its extensive ecosystem of scientific and data analysis libraries. TensorFlow was selected for predictive modeling because it supports **scalable training and deployment of machine learning models**, making it suitable for weather forecasting and extreme event prediction.

Feast (Feature Store)

Feast was selected as the feature store to ensure **feature consistency between model training and inference**. It enables centralized feature management, reduces redundant feature engineering, and improves reproducibility and reliability of machine learning workflows.

FastAPI (API Layer)

FastAPI was chosen for serving analytics and predictions because of its **high performance, lightweight design, and native support for asynchronous processing**. It enables fast and scalable exposure of processed data and ML outputs to dashboards and external consumers.

Redis (Caching Layer)

Redis was selected as the caching layer to **reduce response latency and backend load**. By caching frequently requested aggregates and query results, Redis significantly improves dashboard responsiveness and system scalability.

Tableau (Dashboard / BI Tools)

Tableau was chosen for visualization due to its ability to create **interactive, user-friendly dashboards**. It allows non-technical users to explore weather trends, regional comparisons, and historical insights without direct interaction with backend systems.

Prometheus & Grafana (Monitoring and Observability)

Prometheus was selected for metrics collection due to its **pull-based monitoring model and scalability**, while Grafana was chosen for **visualization and alerting**. Together, they provide real-time visibility into system performance, enabling proactive detection of failures and bottlenecks.

IAM, Encryption, and Audit Logs (Security & Governance)

IAM roles were selected to enforce **role-based access control**, ensuring that only authorized users and services can access sensitive data. Encryption was applied both at rest and in transit to protect data confidentiality. Audit logs were included to support **compliance, traceability, and forensic analysis**, which are critical for enterprise-grade data platforms.

- Task 2 -

Section 01 - Hadoop MapReduce Analysis

Shell Commands Used: Preprocessing

- mvn archetype:generate -DgroupId=validate.weather -DartifactId=preprocessing -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
- mvn clean install
- docker exec -it namenode /bin/bash
- ls -al /opt/hadoop/preprocessing/target
- hdfs dfs -mkdir -p /data/raw
- hdfs dfs -rm -r /preprocessing/validation/weatherData
- hdfs dfs -put /home/data/raw/locationData.csv /data/raw/
- hdfs dfs -put /home/data/raw/weatherData.csv /data/raw/
- hdfs dfs -ls /data/raw
- hdfs dfs -ls /preprocessing/validation/weatherData
- hdfs dfs -cat /preprocessing/validation/weatherData/part-r-00000
- yarn jar /opt/hadoop/preprocessing/target/preprocessing-1.0-SNAPSHOT.jar /data/raw/weatherData.csv /preprocessing/validation/weatherData

Shell Commands Used: Question 1,2

- mvn archetype:generate -DgroupId=analyse.weather -DartifactId=postprocessing -DarchetypeArtifactId=maven-archetype-quickstart -DinteractiveMode=false
- mvn clean install
- docker exec -it namenode /bin/bash
- ls -al /opt/hadoop/task-2/1-mapreduce/part-1/postprocessing/target
- ls -al /opt/hadoop/task-2/1-mapreduce/part-2/postprocessing/target
- hdfs dfs -rm -r /task-2/1-mapreduce/part-1
- hdfs dfs -rm -r /task-2/1-mapreduce/part-2
- hdfs dfs -ls /task-2/1-mapreduce/part-1
- hdfs dfs -ls /task-2/1-mapreduce/part-2
- hdfs dfs -cat /task-2/1-mapreduce/part-1/part-r-00000
- hdfs dfs -cat /task-2/1-mapreduce/part-2/part-r-00000

- yarn jar
/opt/hadoop/task-2/1-mapreduce/part-1/postprocessing/target/weather-analysis-1.0-SNAPSHOT.jar /data/raw/weatherData.csv /task-2/1-mapreduce/part-1 /data/raw/locationData.csv
- yarn jar
/opt/hadoop/task-2/1-mapreduce/part-2/postprocessing/target/weather-analysis-1.0-SNAPSHOT.jar /data/raw/weatherData.csv /task-2/1-mapreduce/part-2

Preprocessing

Outputs/Results:

```
root@ad18e5d711b2:/# hdfs dfs -cat /preprocessing/validation/weatherData/part-r-00000  
2025-12-17 20:00:56,064 INFO sasl.SaslDataTransferClient: SASL encryption trust check:  
localhostTrusted = false, remoteHostTrusted = false  
VALID 142371
```

Code Listings:

Mapper: ValidateMapper.java

```
package validate.weather.mapper;

import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;

import java.io.IOException;

import java.time.LocalDateTime;
import java.time.LocalDate;
import java.time.format.DateTimeFormatter;
import java.time.format.DateTimeParseException;

public class ValidateMapper extends Mapper<LongWritable, Text, Text, Text> {

    private static final DateTimeFormatter TIME_FORMATTER =
DateTimeFormatter.ofPattern("H:mm");
    private static final DateTimeFormatter DATE_FORMATTER =
DateTimeFormatter.ofPattern("M/d/yyyy");

    // Keys emitted: "VALID" or specific error tags like "ERR_location_id_range", etc.
    private final Text outKey = new Text();
    private final Text outVal = new Text("1"); // count = 1

    @Override
    protected void map(LongWritable key, Text value, Context context) throws IOException,
InterruptedException {
        String line = value.toString();

        // skip header (assumes header starts with 'location_id' or is first line)
        if (key.get() == 0 && line.toLowerCase().startsWith("location_id")) {
            return;
        }

        // keep empty tokens
        String[] f = line.split(",", -1);

        if (f.length != 21) {
            outKey.set("ERR_field_count");
            context.write(outKey, outVal);
            return;
        }

        // Validate fields step by step; on any failure emit the specific error and return.
```

```

        // Field idx mapping per your confirmed order:
        // 0 location_id, 1 date, 2 weather_code, 3..8 temps, 9 daylight_duration, 10
sunshine_duration,
        // 11 precipitation_sum,12 rain_sum,13 precipitation_hours,14 wind_speed_10m_max,15
wind_gusts_10m_max,
        // 16 wind_direction_10m_dominant,17 shortwave_radiation_sum,18
et0_fao_evapotranspiration,
        // 19 sunrise,20 sunset

        // location_id
String s;
try {
    s = f[0].trim();
    if (s.isEmpty()) { emitErr("ERR_location_id_null", context); return; }
    int loc = Integer.parseInt(s);
    if (loc < 0 || loc > 26) { emitErr("ERR_location_id_range", context); return; }
} catch (NumberFormatException e) { emitErr("ERR_location_id_type", context);
return; }

// date
s = f[1].trim();
if (s.isEmpty()) { emitErr("ERR_date_null", context); return; }
try {
    LocalDate.parse(s, DATE_FORMATTER);
} catch (DateTimeParseException e) { emitErr("ERR_date_invalid", context); return; }

// weather_code
try {
    s = f[2].trim();
    if (s.isEmpty()) { emitErr("ERR_weather_code_null", context); return; }
    int wc = Integer.parseInt(s);
    if (wc < 0 || wc > 99) { emitErr("ERR_weather_code_range", context); return; }
} catch (NumberFormatException e) { emitErr("ERR_weather_code_type", context);
return; }

// temperature fields (3..8) -> double, -100..100
for (int i = 3; i <= 8; i++) {
    s = f[i].trim();
    if (s.isEmpty()) { emitErr("ERR_temp_null_idx_" + i, context); return; }
    try {
        double d = Double.parseDouble(s);
        if (d < -100.0 || d > 100.0) { emitErr("ERR_temp_range_idx_" + i, context);
return; }
    } catch (NumberFormatException e) { emitErr("ERR_temp_type_idx_" + i, context);
return; }
}

// daylight_duration (9) and sunshine_duration (10): double 0..86400
for (int i : new int[]{9, 10}) {

```

```

        s = f[i].trim();
        if (s.isEmpty()) { emitErr("ERR_duration_null_idx_" + i, context); return; }
        try {
            double d = Double.parseDouble(s);
            if (d < 0.0 || d > 86400.0) { emitErr("ERR_duration_range_idx_" + i,
context); return; }
            } catch (NumberFormatException e) { emitErr("ERR_duration_type_idx_" + i,
context); return; }
        }

// precipitation_sum (11), rain_sum (12): double >= 0
for (int i : new int[]{11, 12}) {
    s = f[i].trim();
    if (s.isEmpty()) { emitErr("ERR_precip_null_idx_" + i, context); return; }
    try {
        double d = Double.parseDouble(s);
        if (d < 0.0) { emitErr("ERR_precip_range_idx_" + i, context); return; }
        } catch (NumberFormatException e) { emitErr("ERR_precip_type_idx_" + i,
context); return; }
    }

// precipitation_hours (13): int 0..24
try {
    s = f[13].trim();
    if (s.isEmpty()) { emitErr("ERR_precip_hours_null", context); return; }
    int ph = Integer.parseInt(s);
    if (ph < 0 || ph > 24) { emitErr("ERR_precip_hours_range", context); return; }
} catch (NumberFormatException e) { emitErr("ERR_precip_hours_type", context);
return; }

// wind_speed_10m_max (14), wind_gusts_10m_max (15): double >= 0
for (int i : new int[]{14, 15}) {
    s = f[i].trim();
    if (s.isEmpty()) { emitErr("ERR_wind_null_idx_" + i, context); return; }
    try {
        double d = Double.parseDouble(s);
        if (d < 0.0) { emitErr("ERR_wind_range_idx_" + i, context); return; }
        } catch (NumberFormatException e) { emitErr("ERR_wind_type_idx_" + i, context);
return; }
    }

// wind_direction_10m_dominant (16): int 0..360
try {
    s = f[16].trim();
    if (s.isEmpty()) { emitErr("ERR_wind_dir_null", context); return; }
    int wd = Integer.parseInt(s);
    if (wd < 0 || wd > 360) { emitErr("ERR_wind_dir_range", context); return; }
} catch (NumberFormatException e) { emitErr("ERR_wind_dir_type", context); return; }

```

```

        // shortwave_radiation_sum (17), et0_fao_evapotranspiration (18): double >=0
        for (int i : new int[]{17, 18}) {
            s = f[i].trim();
            if (s.isEmpty()) { emitErr("ERR_rad_null_idx_" + i, context); return; }
            try {
                double d = Double.parseDouble(s);
                if (d < 0.0) { emitErr("ERR_rad_range_idx_" + i, context); return; }
            } catch (NumberFormatException e) { emitErr("ERR_rad_type_idx_" + i, context);
return; }
        }

        // sunrise (19) and sunset (20): HH:MM 24h
        for (int i : new int[]{19, 20}) {
            s = f[i].trim();
            if (s.isEmpty()) { emitErr("ERR_time_null_idx_" + i, context); return; }
            try {
                LocalTime.parse(s, TIME_FORMATTER);
            } catch (DateTimeParseException e) { emitErr("ERR_time_format_idx_" + i,
context); return; }
        }

        // all checks passed
        outKey.set("VALID");
        context.write(outKey, outVal);
    }

    private void emitErr(String tag, Context context) throws IOException,
InterruptedException {
        outKey.set(tag);
        context.write(outKey, outVal);
    }
}

```

Reducer: ValidateReducer.java

```
package validate.weather.reducer;

import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;

import java.io.IOException;

public class ValidateReducer extends Reducer<Text, Text, Text, IntWritable> {

    private final IntWritable outVal = new IntWritable();

    @Override
    protected void reduce(Text key, Iterable<Text> values, Context context)
        throws IOException, InterruptedException {

        int sum = 0;
        for (Text val : values) {
            // each mapper emitted "1" as Text, so parse it to integer and sum
            sum += Integer.parseInt(val.toString());
        }

        outVal.set(sum);
        context.write(key, outVal); // emit key (error type or VALID) and total count
    }
}
```

Driver: ValidateDriver.java

```
package validate.weather.driver;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.TextInputFormat;
import org.apache.hadoop.mapreduce.lib.output.TextOutputFormat;

import validate.weather.mapper.ValidateMapper;
import validate.weather.reducer.ValidateReducer;

public class ValidateDriver {

    public static void main(String[] args) throws Exception {
        if (args.length != 2) {
            System.err.println("Usage: ValidatorDriver <input-path> <output-path>");
            System.exit(2);
        }

        // Hadoop configuration
        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "Validate Weather Data CSV Schema");

        // Set the driver class (for Hadoop to locate jar)
        job.setJarByClass(ValidateDriver.class);

        // Set mapper and reducer classes
        job.setMapperClass(ValidateMapper.class);
        job.setReducerClass(ValidateReducer.class);

        // Mapper output types
        job.setMapOutputKeyClass(Text.class);
        job.setMapOutputValueClass(Text.class);

        // Reducer output types
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(IntWritable.class);

        // Input and output paths
        TextInputFormat.addInputPath(job, new Path(args[0]));
        TextOutputFormat.setOutputPath(job, new Path(args[1]));

        // Input and output formats
```

```
    job.setInputFormatClass(TextInputFormat.class);  
    job.setOutputFormatClass(TextOutputFormat.class);  
  
    // Run the job and exit  
    System.exit(job.waitForCompletion(true) ? 0 : 1);  
}  
{
```

Question 01

Outputs/Results:

```
root@ad18e5d711b2:/# hdfs dfs -cat /task-2/1-mapreduce/part-1/part-r-00000
```

```
2025-12-17 20:04:06,795 INFO sasl.SaslDataTransferClient: SASL encryption trust check:
```

```
localhostTrusted = false, remoteHostTrusted = false
```

```
Ampara had a total precipitation of 3976 hours with a mean temperature of 24.8 for 1st month
```

```
Ampara had a total precipitation of 3466 hours with a mean temperature of 27.0 for 10th month
```

```
Ampara had a total precipitation of 4752 hours with a mean temperature of 25.7 for 11th month
```

```
Ampara had a total precipitation of 5144 hours with a mean temperature of 25.1 for 12th month
```

```
Ampara had a total precipitation of 3268 hours with a mean temperature of 25.4 for 2nd month
```

```
Ampara had a total precipitation of 2435 hours with a mean temperature of 26.6 for 3rd month
```

```
Ampara had a total precipitation of 2225 hours with a mean temperature of 27.8 for 4th month
```

```
Ampara had a total precipitation of 1775 hours with a mean temperature of 28.6 for 5th month
```

```
Ampara had a total precipitation of 877 hours with a mean temperature of 29.3 for 6th month
```

```
Ampara had a total precipitation of 896 hours with a mean temperature of 29.4 for 7th month
```

```
Ampara had a total precipitation of 1139 hours with a mean temperature of 29.1 for 8th month
```

```
Ampara had a total precipitation of 1555 hours with a mean temperature of 28.4 for 9th month
```

```
Anuradhapura had a total precipitation of 2022 hours with a mean temperature of 24.6 for 1st month
```

```
Anuradhapura had a total precipitation of 3488 hours with a mean temperature of 26.4 for 10th month
```

```
Anuradhapura had a total precipitation of 3995 hours with a mean temperature of 25.3 for 11th month
```

```
Anuradhapura had a total precipitation of 3414 hours with a mean temperature of 24.8 for 12th month
```

```
Anuradhapura had a total precipitation of 1348 hours with a mean temperature of 25.5 for 2nd month
```

```
Anuradhapura had a total precipitation of 1125 hours with a mean temperature of 27.4 for 3rd month
```

```
Anuradhapura had a total precipitation of 1941 hours with a mean temperature of 28.1 for 4th month
```

```
Anuradhapura had a total precipitation of 2276 hours with a mean temperature of 27.8 for 5th month
```

```
Anuradhapura had a total precipitation of 1562 hours with a mean temperature of 27.7 for 6th month
```

```
Anuradhapura had a total precipitation of 1278 hours with a mean temperature of 27.8 for 7th month
```

```
Anuradhapura had a total precipitation of 1538 hours with a mean temperature of 27.6 for 8th month
```

```
Anuradhapura had a total precipitation of 2046 hours with a mean temperature of 27.2 for 9th month
```

```
Badulla had a total precipitation of 4010 hours with a mean temperature of 20.9 for 1st month
```

```
Badulla had a total precipitation of 3517 hours with a mean temperature of 23.4 for 10th month
```

```
Badulla had a total precipitation of 4561 hours with a mean temperature of 22.1 for 11th month
```

```
Badulla had a total precipitation of 4781 hours with a mean temperature of 21.5 for 12th month
```

```
Badulla had a total precipitation of 3286 hours with a mean temperature of 21.6 for 2nd month
```

```
Badulla had a total precipitation of 2416 hours with a mean temperature of 22.8 for 3rd month
```

```
Badulla had a total precipitation of 2684 hours with a mean temperature of 23.8 for 4th month
```

```
Badulla had a total precipitation of 2511 hours with a mean temperature of 24.7 for 5th month
```

```
Badulla had a total precipitation of 1386 hours with a mean temperature of 25.1 for 6th month
```

```
Badulla had a total precipitation of 1346 hours with a mean temperature of 25.2 for 7th month
```

```
Badulla had a total precipitation of 1586 hours with a mean temperature of 25.0 for 8th month
```

```
Badulla had a total precipitation of 2052 hours with a mean temperature of 24.4 for 9th month
```

```
Bandarawela had a total precipitation of 2926 hours with a mean temperature of 18.9 for 1st month
```

```
Bandarawela had a total precipitation of 4390 hours with a mean temperature of 20.4 for 10th month
```

```
Bandarawela had a total precipitation of 4544 hours with a mean temperature of 19.7 for 11th month
```

```
Bandarawela had a total precipitation of 4224 hours with a mean temperature of 19.3 for 12th month
```

```
Bandarawela had a total precipitation of 2370 hours with a mean temperature of 19.4 for 2nd month
```

```
Bandarawela had a total precipitation of 2199 hours with a mean temperature of 20.4 for 3rd month
```

```
Bandarawela had a total precipitation of 3083 hours with a mean temperature of 20.9 for 4th month
```

```
Bandarawela had a total precipitation of 3722 hours with a mean temperature of 21.5 for 5th month
```

```
Bandarawela had a total precipitation of 3265 hours with a mean temperature of 21.7 for 6th month
```

```
Bandarawela had a total precipitation of 3162 hours with a mean temperature of 21.7 for 7th month
```

```
Bandarawela had a total precipitation of 3351 hours with a mean temperature of 21.4 for 8th month
```

Bandarawela had a total precipitation of 3541 hours with a mean temperature of 21.0 for 9th month
Batticaloa had a total precipitation of 4567 hours with a mean temperature of 25.4 for 1st month
Batticaloa had a total precipitation of 3631 hours with a mean temperature of 27.2 for 10th month
Batticaloa had a total precipitation of 5308 hours with a mean temperature of 26.1 for 11th month
Batticaloa had a total precipitation of 5741 hours with a mean temperature of 25.6 for 12th month
Batticaloa had a total precipitation of 3566 hours with a mean temperature of 26.0 for 2nd month
Batticaloa had a total precipitation of 2443 hours with a mean temperature of 27.1 for 3rd month
Batticaloa had a total precipitation of 2020 hours with a mean temperature of 28.3 for 4th month
Batticaloa had a total precipitation of 1772 hours with a mean temperature of 29.0 for 5th month
Batticaloa had a total precipitation of 863 hours with a mean temperature of 29.7 for 6th month
Batticaloa had a total precipitation of 885 hours with a mean temperature of 29.6 for 7th month
Batticaloa had a total precipitation of 1058 hours with a mean temperature of 29.2 for 8th month
Batticaloa had a total precipitation of 1588 hours with a mean temperature of 28.4 for 9th month
Colombo had a total precipitation of 1657 hours with a mean temperature of 26.1 for 1st month
Colombo had a total precipitation of 6208 hours with a mean temperature of 26.1 for 10th month
Colombo had a total precipitation of 4859 hours with a mean temperature of 25.8 for 11th month
Colombo had a total precipitation of 3189 hours with a mean temperature of 25.7 for 12th month
Colombo had a total precipitation of 1509 hours with a mean temperature of 26.8 for 2nd month
Colombo had a total precipitation of 2500 hours with a mean temperature of 27.4 for 3rd month
Colombo had a total precipitation of 4092 hours with a mean temperature of 27.5 for 4th month
Colombo had a total precipitation of 7202 hours with a mean temperature of 27.3 for 5th month
Colombo had a total precipitation of 7196 hours with a mean temperature of 26.9 for 6th month
Colombo had a total precipitation of 6018 hours with a mean temperature of 26.7 for 7th month
Colombo had a total precipitation of 6422 hours with a mean temperature of 26.5 for 8th month
Colombo had a total precipitation of 6707 hours with a mean temperature of 26.3 for 9th month
Galle had a total precipitation of 2514 hours with a mean temperature of 25.7 for 1st month
Galle had a total precipitation of 6049 hours with a mean temperature of 26.2 for 10th month
Galle had a total precipitation of 4935 hours with a mean temperature of 26.0 for 11th month
Galle had a total precipitation of 3754 hours with a mean temperature of 25.8 for 12th month
Galle had a total precipitation of 2334 hours with a mean temperature of 26.3 for 2nd month
Galle had a total precipitation of 2824 hours with a mean temperature of 27.0 for 3rd month
Galle had a total precipitation of 4275 hours with a mean temperature of 27.3 for 4th month
Galle had a total precipitation of 7071 hours with a mean temperature of 27.2 for 5th month
Galle had a total precipitation of 6938 hours with a mean temperature of 26.9 for 6th month
Galle had a total precipitation of 5975 hours with a mean temperature of 26.7 for 7th month
Galle had a total precipitation of 6486 hours with a mean temperature of 26.5 for 8th month
Galle had a total precipitation of 6532 hours with a mean temperature of 26.3 for 9th month
Gampaha had a total precipitation of 1465 hours with a mean temperature of 26.1 for 1st month
Gampaha had a total precipitation of 6144 hours with a mean temperature of 25.9 for 10th month
Gampaha had a total precipitation of 4551 hours with a mean temperature of 25.6 for 11th month
Gampaha had a total precipitation of 3047 hours with a mean temperature of 25.6 for 12th month
Gampaha had a total precipitation of 1335 hours with a mean temperature of 27.0 for 2nd month
Gampaha had a total precipitation of 2369 hours with a mean temperature of 27.5 for 3rd month
Gampaha had a total precipitation of 3929 hours with a mean temperature of 27.3 for 4th month
Gampaha had a total precipitation of 6817 hours with a mean temperature of 27.0 for 5th month
Gampaha had a total precipitation of 7410 hours with a mean temperature of 26.6 for 6th month
Gampaha had a total precipitation of 6217 hours with a mean temperature of 26.5 for 7th month
Gampaha had a total precipitation of 6334 hours with a mean temperature of 26.2 for 8th month
Gampaha had a total precipitation of 6636 hours with a mean temperature of 26.0 for 9th month
Hambantota had a total precipitation of 1738 hours with a mean temperature of 26.2 for 1st month
Hambantota had a total precipitation of 3336 hours with a mean temperature of 27.1 for 10th month
Hambantota had a total precipitation of 4047 hours with a mean temperature of 26.3 for 11th month
Hambantota had a total precipitation of 3224 hours with a mean temperature of 26.0 for 12th month
Hambantota had a total precipitation of 1394 hours with a mean temperature of 26.8 for 2nd month

Hambantota had a total precipitation of 1941 hours with a mean temperature of 27.6 for 3rd month
Hambantota had a total precipitation of 2796 hours with a mean temperature of 28.0 for 4th month
Hambantota had a total precipitation of 2929 hours with a mean temperature of 28.1 for 5th month
Hambantota had a total precipitation of 1786 hours with a mean temperature of 28.1 for 6th month
Hambantota had a total precipitation of 1397 hours with a mean temperature of 28.2 for 7th month
Hambantota had a total precipitation of 1769 hours with a mean temperature of 28.0 for 8th month
Hambantota had a total precipitation of 2495 hours with a mean temperature of 27.5 for 9th month
Jaffna had a total precipitation of 1724 hours with a mean temperature of 25.5 for 1st month
Jaffna had a total precipitation of 4201 hours with a mean temperature of 27.5 for 10th month
Jaffna had a total precipitation of 5121 hours with a mean temperature of 26.4 for 11th month
Jaffna had a total precipitation of 3897 hours with a mean temperature of 25.8 for 12th month
Jaffna had a total precipitation of 1000 hours with a mean temperature of 26.3 for 2nd month
Jaffna had a total precipitation of 840 hours with a mean temperature of 28.0 for 3rd month
Jaffna had a total precipitation of 1847 hours with a mean temperature of 29.3 for 4th month
Jaffna had a total precipitation of 1967 hours with a mean temperature of 29.4 for 5th month
Jaffna had a total precipitation of 1137 hours with a mean temperature of 29.1 for 6th month
Jaffna had a total precipitation of 1197 hours with a mean temperature of 28.8 for 7th month
Jaffna had a total precipitation of 1673 hours with a mean temperature of 28.4 for 8th month
Jaffna had a total precipitation of 1995 hours with a mean temperature of 28.1 for 9th month
Kalutara had a total precipitation of 1794 hours with a mean temperature of 25.9 for 1st month
Kalutara had a total precipitation of 6255 hours with a mean temperature of 26.1 for 10th month
Kalutara had a total precipitation of 4844 hours with a mean temperature of 25.8 for 11th month
Kalutara had a total precipitation of 3328 hours with a mean temperature of 25.7 for 12th month
Kalutara had a total precipitation of 1686 hours with a mean temperature of 26.5 for 2nd month
Kalutara had a total precipitation of 2500 hours with a mean temperature of 27.1 for 3rd month
Kalutara had a total precipitation of 4137 hours with a mean temperature of 27.3 for 4th month
Kalutara had a total precipitation of 7274 hours with a mean temperature of 27.1 for 5th month
Kalutara had a total precipitation of 7205 hours with a mean temperature of 26.8 for 6th month
Kalutara had a total precipitation of 6088 hours with a mean temperature of 26.6 for 7th month
Kalutara had a total precipitation of 6442 hours with a mean temperature of 26.4 for 8th month
Kalutara had a total precipitation of 6707 hours with a mean temperature of 26.2 for 9th month
Kandy had a total precipitation of 1675 hours with a mean temperature of 22.7 for 1st month
Kandy had a total precipitation of 4446 hours with a mean temperature of 23.4 for 10th month
Kandy had a total precipitation of 4025 hours with a mean temperature of 23.0 for 11th month
Kandy had a total precipitation of 3330 hours with a mean temperature of 22.7 for 12th month
Kandy had a total precipitation of 1381 hours with a mean temperature of 23.6 for 2nd month
Kandy had a total precipitation of 1601 hours with a mean temperature of 25.0 for 3rd month
Kandy had a total precipitation of 2825 hours with a mean temperature of 25.2 for 4th month
Kandy had a total precipitation of 3888 hours with a mean temperature of 24.7 for 5th month
Kandy had a total precipitation of 4169 hours with a mean temperature of 24.2 for 6th month
Kandy had a total precipitation of 4074 hours with a mean temperature of 23.9 for 7th month
Kandy had a total precipitation of 4011 hours with a mean temperature of 23.7 for 8th month
Kandy had a total precipitation of 3456 hours with a mean temperature of 23.6 for 9th month
Kegalle had a total precipitation of 1404 hours with a mean temperature of 24.8 for 1st month
Kegalle had a total precipitation of 5357 hours with a mean temperature of 24.6 for 10th month
Kegalle had a total precipitation of 3974 hours with a mean temperature of 24.5 for 11th month
Kegalle had a total precipitation of 2904 hours with a mean temperature of 24.4 for 12th month
Kegalle had a total precipitation of 1071 hours with a mean temperature of 25.9 for 2nd month
Kegalle had a total precipitation of 1628 hours with a mean temperature of 26.9 for 3rd month
Kegalle had a total precipitation of 2925 hours with a mean temperature of 26.4 for 4th month
Kegalle had a total precipitation of 5090 hours with a mean temperature of 25.9 for 5th month
Kegalle had a total precipitation of 6104 hours with a mean temperature of 25.3 for 6th month
Kegalle had a total precipitation of 5590 hours with a mean temperature of 25.1 for 7th month
Kegalle had a total precipitation of 5623 hours with a mean temperature of 24.9 for 8th month

Kegalle had a total precipitation of 5321 hours with a mean temperature of 24.8 for 9th month
Kilinochchi[1] had a total precipitation of 2381 hours with a mean temperature of 25.2 for 1st month
Kilinochchi[1] had a total precipitation of 3487 hours with a mean temperature of 27.4 for 10th month
Kilinochchi[1] had a total precipitation of 5137 hours with a mean temperature of 26.1 for 11th month
Kilinochchi[1] had a total precipitation of 4430 hours with a mean temperature of 25.5 for 12th month
Kilinochchi[1] had a total precipitation of 1474 hours with a mean temperature of 25.9 for 2nd month
Kilinochchi[1] had a total precipitation of 970 hours with a mean temperature of 27.6 for 3rd month
Kilinochchi[1] had a total precipitation of 1342 hours with a mean temperature of 29.2 for 4th month
Kilinochchi[1] had a total precipitation of 1247 hours with a mean temperature of 29.4 for 5th month
Kilinochchi[1] had a total precipitation of 591 hours with a mean temperature of 29.7 for 6th month
Kilinochchi[1] had a total precipitation of 718 hours with a mean temperature of 29.4 for 7th month
Kilinochchi[1] had a total precipitation of 1030 hours with a mean temperature of 29.0 for 8th month
Kilinochchi[1] had a total precipitation of 1279 hours with a mean temperature of 28.7 for 9th month
Kurunegala had a total precipitation of 1399 hours with a mean temperature of 25.0 for 1st month
Kurunegala had a total precipitation of 4779 hours with a mean temperature of 25.2 for 10th month
Kurunegala had a total precipitation of 3792 hours with a mean temperature of 25.0 for 11th month
Kurunegala had a total precipitation of 2879 hours with a mean temperature of 24.8 for 12th month
Kurunegala had a total precipitation of 1039 hours with a mean temperature of 26.1 for 2nd month
Kurunegala had a total precipitation of 1411 hours with a mean temperature of 27.5 for 3rd month
Kurunegala had a total precipitation of 2706 hours with a mean temperature of 27.2 for 4th month
Kurunegala had a total precipitation of 4404 hours with a mean temperature of 26.5 for 5th month
Kurunegala had a total precipitation of 5036 hours with a mean temperature of 25.9 for 6th month
Kurunegala had a total precipitation of 4449 hours with a mean temperature of 25.8 for 7th month
Kurunegala had a total precipitation of 4515 hours with a mean temperature of 25.7 for 8th month
Kurunegala had a total precipitation of 4457 hours with a mean temperature of 25.4 for 9th month
Mannar had a total precipitation of 1780 hours with a mean temperature of 25.6 for 1st month
Mannar had a total precipitation of 3808 hours with a mean temperature of 27.3 for 10th month
Mannar had a total precipitation of 4452 hours with a mean temperature of 26.2 for 11th month
Mannar had a total precipitation of 3377 hours with a mean temperature of 25.8 for 12th month
Mannar had a total precipitation of 1058 hours with a mean temperature of 26.5 for 2nd month
Mannar had a total precipitation of 990 hours with a mean temperature of 28.2 for 3rd month
Mannar had a total precipitation of 2102 hours with a mean temperature of 29.1 for 4th month
Mannar had a total precipitation of 2156 hours with a mean temperature of 29.1 for 5th month
Mannar had a total precipitation of 1196 hours with a mean temperature of 28.9 for 6th month
Mannar had a total precipitation of 929 hours with a mean temperature of 28.5 for 7th month
Mannar had a total precipitation of 1158 hours with a mean temperature of 28.2 for 8th month
Mannar had a total precipitation of 1778 hours with a mean temperature of 28.0 for 9th month
Matale had a total precipitation of 1733 hours with a mean temperature of 23.5 for 1st month
Matale had a total precipitation of 4544 hours with a mean temperature of 24.5 for 10th month
Matale had a total precipitation of 4046 hours with a mean temperature of 24.0 for 11th month
Matale had a total precipitation of 3306 hours with a mean temperature of 23.6 for 12th month
Matale had a total precipitation of 1328 hours with a mean temperature of 24.4 for 2nd month
Matale had a total precipitation of 1532 hours with a mean temperature of 26.0 for 3rd month
Matale had a total precipitation of 2754 hours with a mean temperature of 26.2 for 4th month
Matale had a total precipitation of 3898 hours with a mean temperature of 25.7 for 5th month
Matale had a total precipitation of 4369 hours with a mean temperature of 25.3 for 6th month
Matale had a total precipitation of 4065 hours with a mean temperature of 25.1 for 7th month
Matale had a total precipitation of 3989 hours with a mean temperature of 25.0 for 8th month
Matale had a total precipitation of 3738 hours with a mean temperature of 24.8 for 9th month
Matara had a total precipitation of 2109 hours with a mean temperature of 25.9 for 1st month
Matara had a total precipitation of 5457 hours with a mean temperature of 26.3 for 10th month
Matara had a total precipitation of 4642 hours with a mean temperature of 26.2 for 11th month
Matara had a total precipitation of 3393 hours with a mean temperature of 26.0 for 12th month
Matara had a total precipitation of 1846 hours with a mean temperature of 26.5 for 2nd month

Matara had a total precipitation of 2348 hours with a mean temperature of 27.2 for 3rd month
Matara had a total precipitation of 3776 hours with a mean temperature of 27.5 for 4th month
Matara had a total precipitation of 6345 hours with a mean temperature of 27.3 for 5th month
Matara had a total precipitation of 6045 hours with a mean temperature of 27.0 for 6th month
Matara had a total precipitation of 5126 hours with a mean temperature of 26.7 for 7th month
Matara had a total precipitation of 5440 hours with a mean temperature of 26.5 for 8th month
Matara had a total precipitation of 5947 hours with a mean temperature of 26.4 for 9th month
Moneragala had a total precipitation of 2825 hours with a mean temperature of 24.3 for 1st month
Moneragala had a total precipitation of 3263 hours with a mean temperature of 26.2 for 10th month
Moneragala had a total precipitation of 4244 hours with a mean temperature of 24.8 for 11th month
Moneragala had a total precipitation of 4162 hours with a mean temperature of 24.4 for 12th month
Moneragala had a total precipitation of 2325 hours with a mean temperature of 24.9 for 2nd month
Moneragala had a total precipitation of 2191 hours with a mean temperature of 26.0 for 3rd month
Moneragala had a total precipitation of 2393 hours with a mean temperature of 26.7 for 4th month
Moneragala had a total precipitation of 1900 hours with a mean temperature of 27.3 for 5th month
Moneragala had a total precipitation of 1003 hours with a mean temperature of 28.2 for 6th month
Moneragala had a total precipitation of 991 hours with a mean temperature of 28.2 for 7th month
Moneragala had a total precipitation of 1188 hours with a mean temperature of 28.0 for 8th month
Moneragala had a total precipitation of 1540 hours with a mean temperature of 27.5 for 9th month
Mullaitivu had a total precipitation of 3165 hours with a mean temperature of 25.6 for 1st month
Mullaitivu had a total precipitation of 3434 hours with a mean temperature of 27.4 for 10th month
Mullaitivu had a total precipitation of 5537 hours with a mean temperature of 26.2 for 11th month
Mullaitivu had a total precipitation of 5114 hours with a mean temperature of 25.8 for 12th month
Mullaitivu had a total precipitation of 2040 hours with a mean temperature of 26.1 for 2nd month
Mullaitivu had a total precipitation of 1263 hours with a mean temperature of 27.4 for 3rd month
Mullaitivu had a total precipitation of 1311 hours with a mean temperature of 28.7 for 4th month
Mullaitivu had a total precipitation of 1290 hours with a mean temperature of 29.3 for 5th month
Mullaitivu had a total precipitation of 611 hours with a mean temperature of 29.8 for 6th month
Mullaitivu had a total precipitation of 782 hours with a mean temperature of 29.4 for 7th month
Mullaitivu had a total precipitation of 1078 hours with a mean temperature of 29.0 for 8th month
Mullaitivu had a total precipitation of 1288 hours with a mean temperature of 28.6 for 9th month
Nuwara Eliya had a total precipitation of 2745 hours with a mean temperature of 15.8 for 1st month
Nuwara Eliya had a total precipitation of 5027 hours with a mean temperature of 16.5 for 10th month
Nuwara Eliya had a total precipitation of 4402 hours with a mean temperature of 16.3 for 11th month
Nuwara Eliya had a total precipitation of 4212 hours with a mean temperature of 16.0 for 12th month
Nuwara Eliya had a total precipitation of 2190 hours with a mean temperature of 16.3 for 2nd month
Nuwara Eliya had a total precipitation of 2051 hours with a mean temperature of 17.0 for 3rd month
Nuwara Eliya had a total precipitation of 3059 hours with a mean temperature of 17.4 for 4th month
Nuwara Eliya had a total precipitation of 4768 hours with a mean temperature of 17.6 for 5th month
Nuwara Eliya had a total precipitation of 5212 hours with a mean temperature of 17.0 for 6th month
Nuwara Eliya had a total precipitation of 4944 hours with a mean temperature of 16.7 for 7th month
Nuwara Eliya had a total precipitation of 4984 hours with a mean temperature of 16.6 for 8th month
Nuwara Eliya had a total precipitation of 4646 hours with a mean temperature of 16.6 for 9th month
Polonnaruwa had a total precipitation of 3161 hours with a mean temperature of 24.6 for 1st month
Polonnaruwa had a total precipitation of 2952 hours with a mean temperature of 27.2 for 10th month
Polonnaruwa had a total precipitation of 4139 hours with a mean temperature of 25.5 for 11th month
Polonnaruwa had a total precipitation of 4401 hours with a mean temperature of 24.9 for 12th month
Polonnaruwa had a total precipitation of 2395 hours with a mean temperature of 25.3 for 2nd month
Polonnaruwa had a total precipitation of 1678 hours with a mean temperature of 26.9 for 3rd month
Polonnaruwa had a total precipitation of 1566 hours with a mean temperature of 28.3 for 4th month
Polonnaruwa had a total precipitation of 1427 hours with a mean temperature of 28.8 for 5th month
Polonnaruwa had a total precipitation of 569 hours with a mean temperature of 29.3 for 6th month
Polonnaruwa had a total precipitation of 617 hours with a mean temperature of 29.1 for 7th month
Polonnaruwa had a total precipitation of 794 hours with a mean temperature of 29.0 for 8th month

Polonnaruwa had a total precipitation of 1089 hours with a mean temperature of 28.6 for 9th month
Puttalam had a total precipitation of 1470 hours with a mean temperature of 25.6 for 1st month
Puttalam had a total precipitation of 4920 hours with a mean temperature of 26.7 for 10th month
Puttalam had a total precipitation of 3975 hours with a mean temperature of 26.1 for 11th month
Puttalam had a total precipitation of 2798 hours with a mean temperature of 25.7 for 12th month
Puttalam had a total precipitation of 1022 hours with a mean temperature of 26.5 for 2nd month
Puttalam had a total precipitation of 1491 hours with a mean temperature of 27.8 for 3rd month
Puttalam had a total precipitation of 3116 hours with a mean temperature of 28.1 for 4th month
Puttalam had a total precipitation of 4610 hours with a mean temperature of 28.0 for 5th month
Puttalam had a total precipitation of 4229 hours with a mean temperature of 27.7 for 6th month
Puttalam had a total precipitation of 2762 hours with a mean temperature of 27.5 for 7th month
Puttalam had a total precipitation of 3227 hours with a mean temperature of 27.3 for 8th month
Puttalam had a total precipitation of 4298 hours with a mean temperature of 27.1 for 9th month
Ratnapura had a total precipitation of 2323 hours with a mean temperature of 25.0 for 1st month
Ratnapura had a total precipitation of 6264 hours with a mean temperature of 24.9 for 10th month
Ratnapura had a total precipitation of 4784 hours with a mean temperature of 24.8 for 11th month
Ratnapura had a total precipitation of 3647 hours with a mean temperature of 24.7 for 12th month
Ratnapura had a total precipitation of 2059 hours with a mean temperature of 25.6 for 2nd month
Ratnapura had a total precipitation of 2553 hours with a mean temperature of 26.2 for 3rd month
Ratnapura had a total precipitation of 4230 hours with a mean temperature of 26.2 for 4th month
Ratnapura had a total precipitation of 7365 hours with a mean temperature of 25.9 for 5th month
Ratnapura had a total precipitation of 7899 hours with a mean temperature of 25.4 for 6th month
Ratnapura had a total precipitation of 7266 hours with a mean temperature of 25.2 for 7th month
Ratnapura had a total precipitation of 7403 hours with a mean temperature of 25.0 for 8th month
Ratnapura had a total precipitation of 7127 hours with a mean temperature of 24.9 for 9th month
Trincomalee had a total precipitation of 3909 hours with a mean temperature of 25.6 for 1st month
Trincomalee had a total precipitation of 3481 hours with a mean temperature of 27.4 for 10th month
Trincomalee had a total precipitation of 5456 hours with a mean temperature of 26.2 for 11th month
Trincomalee had a total precipitation of 5504 hours with a mean temperature of 25.8 for 12th month
Trincomalee had a total precipitation of 2689 hours with a mean temperature of 26.2 for 2nd month
Trincomalee had a total precipitation of 1864 hours with a mean temperature of 27.4 for 3rd month
Trincomalee had a total precipitation of 1659 hours with a mean temperature of 28.7 for 4th month
Trincomalee had a total precipitation of 1502 hours with a mean temperature of 29.2 for 5th month
Trincomalee had a total precipitation of 629 hours with a mean temperature of 29.7 for 6th month
Trincomalee had a total precipitation of 751 hours with a mean temperature of 29.3 for 7th month
Trincomalee had a total precipitation of 983 hours with a mean temperature of 28.9 for 8th month
Trincomalee had a total precipitation of 1350 hours with a mean temperature of 28.5 for 9th month
Vavuniya had a total precipitation of 2111 hours with a mean temperature of 24.4 for 1st month
Vavuniya had a total precipitation of 3348 hours with a mean temperature of 26.6 for 10th month
Vavuniya had a total precipitation of 4076 hours with a mean temperature of 25.2 for 11th month
Vavuniya had a total precipitation of 3697 hours with a mean temperature of 24.7 for 12th month
Vavuniya had a total precipitation of 1357 hours with a mean temperature of 25.3 for 2nd month
Vavuniya had a total precipitation of 1069 hours with a mean temperature of 27.2 for 3rd month
Vavuniya had a total precipitation of 1678 hours with a mean temperature of 28.3 for 4th month
Vavuniya had a total precipitation of 1847 hours with a mean temperature of 28.3 for 5th month
Vavuniya had a total precipitation of 1052 hours with a mean temperature of 28.7 for 6th month
Vavuniya had a total precipitation of 990 hours with a mean temperature of 28.5 for 7th month
Vavuniya had a total precipitation of 1261 hours with a mean temperature of 28.2 for 8th month
Vavuniya had a total precipitation of 1604 hours with a mean temperature of 27.8 for 9th month
Welimada had a total precipitation of 3664 hours with a mean temperature of 19.1 for 1st month
Welimada had a total precipitation of 4288 hours with a mean temperature of 21.2 for 10th month
Welimada had a total precipitation of 4582 hours with a mean temperature of 20.2 for 11th month
Welimada had a total precipitation of 4583 hours with a mean temperature of 19.6 for 12th month
Welimada had a total precipitation of 2942 hours with a mean temperature of 19.6 for 2nd month

Welimada had a total precipitation of 2255 hours with a mean temperature of 20.7 for 3rd month
Welimada had a total precipitation of 2960 hours with a mean temperature of 21.5 for 4th month
Welimada had a total precipitation of 3536 hours with a mean temperature of 22.4 for 5th month
Welimada had a total precipitation of 3076 hours with a mean temperature of 22.7 for 6th month
Welimada had a total precipitation of 3133 hours with a mean temperature of 22.7 for 7th month
Welimada had a total precipitation of 3216 hours with a mean temperature of 22.5 for 8th month
Welimada had a total precipitation of 3315 hours with a mean temperature of 22.0 for 9th month

Code Listings:

Mapper: AnalyseMapper.java

```
package analyse.weather.mapper;

import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;

import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;
import java.net.URI;
import java.util.HashMap;
import java.util.Map;

public class AnalyseMapper extends Mapper<LongWritable, Text, Text, Text> {

    private final Map<String, String> locMap = new HashMap<>();

    @Override
    protected void setup(Context context) throws IOException {
        URI[] cacheFiles = context.getCacheFiles();
        if (cacheFiles != null) {
            for (URI uri : cacheFiles) {
                String path = uri.getPath();
                if (path.endsWith("locationData.csv")) {
                    try (BufferedReader br = new BufferedReader(new
FileReader("locationData.csv"))) {
                        String line;
                        while ((line = br.readLine()) != null) {
                            if (line.toLowerCase().startsWith("location_id")) continue;
                            String[] f = line.split(",", -1);
                            String location_id = f[0].trim();
                            String location = f[7].trim();
                            locMap.put(location_id, location);
                        }
                    }
                }
            }
        }
    }

    @Override
    protected void map(LongWritable key, Text value, Context context) throws IOException,
InterruptedException {
        String line = value.toString();
    }
}
```

```
if (key.get() == 0 && line.toLowerCase().startsWith("location_id")) {  
    return;  
}  
String[] f = line.split(",", -1);  
  
String location_id = f[0].trim();  
String date = f[1].trim();  
String temperature_2m_mean = f[5].trim();  
String precipitation_hours = f[13].trim();  
  
String location = locMap.getOrDefault(location_id, "UNKNOWN");  
String month = date.split("/")[0];  
  
Text outKey = new Text(location + "-" + month);  
Text outVal = new Text(temperature_2m_mean + "," + precipitation_hours);  
  
context.write(outKey, outVal);  
}  
}
```

Reducer: AnalyseReducer.java

```
package analyse.weather.reducer;

import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;

import java.io.IOException;

public class AnalyseReducer extends Reducer<Text, Text, Text, Text> {

    private final Text outVal = new Text();

    @Override
    protected void reduce(Text key, Iterable<Text> values, Context context)
        throws IOException, InterruptedException {

        double sum_temperature_2m_mean = 0.0;
        int sum_precipitation_hours = 0;
        int val_count = 0;

        for (Text val : values) {
            double temperature_2m_mean = Double.parseDouble(val.toString().split(",",
-1)[0]);
            int precipitation_hours = Integer.parseInt(val.toString().split(",", -1)[1]);

            sum_temperature_2m_mean += temperature_2m_mean;
            sum_precipitation_hours += precipitation_hours;

            val_count++;
        }

        double avg_temperature_2m_mean = sum_temperature_2m_mean/val_count;

        String location = key.toString().split("-", -1)[0];
        int month = Integer.parseInt(key.toString().split("-", -1)[1]);

        String monthOrdinal = getMonthOrdinal(month);

        String result = String.format(
            "%s had a total precipitation of %d hours with a mean temperature of %.1f for %s
month",
            location, sum_precipitation_hours, avg_temperature_2m_mean, monthOrdinal
        );

        outVal.set(result);
        context.write(null, outVal);
    }
}
```

```
private String getMonthOrdinal(int month) {  
    if (month > 3) {  
        return month + "th";  
    }  
    switch (month) {  
        case 1: return month + "st";  
        case 2: return month + "nd";  
        case 3: return month + "rd";  
        default: return month + "th";  
    }  
}  
}
```

Driver: AnalyseDriver.java

```
package analyse.weather.driver;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

import analyse.weather.mapper.AnalyseMapper;
import analyse.weather.reducer.AnalyseReducer;

public class AnalyseDriver {

    public static void main(String[] args) throws Exception {

        if (args.length != 3) {
            System.err.println("Usage: AnalyseDriver <input path> <output path> <location  
data csv path>");
            System.exit(-1);
        }

        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "Weather Analysis");
        job.setJarByClass(AnalyseDriver.class);

        // Mapper & Reducer
        job.setMapperClass(AnalyseMapper.class);
        job.setReducerClass(AnalyseReducer.class);

        // Output types
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(Text.class);

        // Input / Output paths
        FileInputFormat.addInputPath(job, new Path(args[0])); // weather CSV
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        // Add the location CSV as a cached file for the Mapper
        job.addCacheFile(new Path(args[2]).toUri()); // locationData.csv

        // Run job
        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

Question 02

Outputs/Results:

```
root@ad18e5d711b2:/# hdfs dfs -cat /task-2/1-mapreduce/part-2/part-r-00000
2025-12-17 20:05:51,068 INFO sasl.SaslDataTransferClient: SASL encryption trust check:
localhostTrusted = false, remoteHostTrusted = false
10th month in 2018 had the total precipitation of 11007 hr
```

Code Listings:

Mapper: AnalyseMapper.java

```
package analyse.weather.mapper;

import org.apache.hadoop.io.LongWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Mapper;

import java.io.BufferedReader;
import java.io.FileReader;
import java.io.IOException;
import java.net.URI;
import java.util.HashMap;
import java.util.Map;

public class AnalyseMapper extends Mapper<LongWritable, Text, Text, Text> {

    @Override
    protected void map(LongWritable key, Text value, Context context) throws IOException,
        InterruptedException {
        String line = value.toString();
        if (key.get() == 0 && line.toLowerCase().startsWith("location_id")) {
            return;
        }
        String[] f = line.split(",", -1);

        String date = f[1].trim();
        String precipitation_hours = f[13].trim();

        String month = date.split("/")[0];
        String year = date.split("/")[2];

        Text outKey = new Text(month + "-" + year);
        Text outVal = new Text(precipitation_hours);

        context.write(outKey, outVal);
    }
}
```

Reducer: AnalyseReducer.java

```
package analyse.weather.reducer;

import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Reducer;

import java.io.IOException;

public class AnalyseReducer extends Reducer<Text, Text, Text, Text> {

    private int max_sum_precipitation_hours = 0;
    private String max_month_year = "";

    @Override
    protected void reduce(Text key, Iterable<Text> values, Context context)
        throws IOException, InterruptedException {

        int sum_precipitation_hours = 0;

        for (Text val : values) {
            int precipitation_hours = Integer.parseInt(val.toString());

            sum_precipitation_hours += precipitation_hours;
        }

        if (sum_precipitation_hours > max_sum_precipitation_hours) {
            max_sum_precipitation_hours = sum_precipitation_hours;
            max_month_year = key.toString();
        }
    }

    @Override
    protected void cleanup(Context context)
        throws IOException, InterruptedException {

        int month = Integer.parseInt(max_month_year.toString().split("-", -1)[0]);
        String year = max_month_year.toString().split("-", -1)[1];

        String monthOrdinal = getMonthOrdinal(month);

        String result = String.format(
            "%s month in %s had the total precipitation of %d hr",
            monthOrdinal, year, max_sum_precipitation_hours
        );

        context.write(null, new Text(result));
    }
}
```

```
}

private String getMonthOrdinal(int month) {
    if (month > 3) {
        return month + "th";
    }
    switch (month) {
        case 1: return month + "st";
        case 2: return month + "nd";
        case 3: return month + "rd";
        default: return month + "th";
    }
}
}
```

Driver: AnalyseDriver.java

```
package analyse.weather.driver;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

import analyse.weather.mapper.AnalyseMapper;
import analyse.weather.reducer.AnalyseReducer;

public class AnalyseDriver {

    public static void main(String[] args) throws Exception {

        if (args.length != 2) {
            System.err.println("Usage: AnalyseDriver <input path> <output path>");
            System.exit(-1);
        }

        Configuration conf = new Configuration();
        Job job = Job.getInstance(conf, "Weather Analysis");
        job.setJarByClass(AnalyseDriver.class);

        // Mapper & Reducer
        job.setMapperClass(AnalyseMapper.class);
        job.setReducerClass(AnalyseReducer.class);

        // Output types
        job.setOutputKeyClass(Text.class);
        job.setOutputValueClass(Text.class);

        // Input / Output paths
        FileInputFormat.addInputPath(job, new Path(args[0])); // weather CSV
        FileOutputFormat.setOutputPath(job, new Path(args[1]));

        // Run job
        System.exit(job.waitForCompletion(true) ? 0 : 1);
    }
}
```

Section 02 - Hive Analysis

Shell Commands Used:

- `docker compose -f docker-compose-fresh.yml up -d`
- `docker exec -it hive-server /bin/bash`
- `hdfs dfs -mkdir -p /data/raw/location`
- `hdfs dfs -mkdir -p /data/raw/weather`
- `hdfs dfs -put /home/data/raw/locationData.csv /data/raw/location`
- `hdfs dfs -put /home/data/raw/weatherData.csv /data/raw/weather`
- `hdfs dfs -ls /data/raw/location`
- `hdfs dfs -ls /data/raw/weather`
- `hive`

Question 01

Outputs/Results:

```
hive> SELECT
>   l.city_name,
>   SUM(w.temperature_2m_max) AS temperature_2m_max_sum
> FROM weatherData w
> JOIN locationData l
>   ON w.location_id = l.location_id
> GROUP BY l.city_name
> ORDER BY temperature_2m_max_sum ASC
> LIMIT 10;
```

Total MapReduce CPU Time Spent: 0 msec

OK

Nuwara Eliya 106839.80000000012

Bandarawela 129607.59999999971

Welimada 133323.29999999987

Badulla 145739.70000000003

Kandy 146239.19999999996

Matara 150293.10000000006

Galle 151652.29999999998

Ratnapura 152115.69999999995

Matale 152280.20000000027

Kalutara 153856.59999999986

Time taken: 10.254 seconds, Fetched: 10 row(s)

Code Listings: Preprocessing

create locationData table.sql

```
CREATE EXTERNAL TABLE locationData (  
  location_id INT,  
  latitude DOUBLE,  
  longitude DOUBLE,  
  elevation INT,  
  utc_offset_seconds INT,  
  timezone STRING,  
  timezone_abbreviation INT,  
  city_name STRING  
)  
ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.OpenCSVSerde'  
WITH SERDEPROPERTIES ("separatorChar" = ",")  
STORED AS TEXTFILE  
LOCATION '/data/raw/location'  
TBLPROPERTIES ("skip.header.line.count"="1");
```

create raw weatherData table.sql

```
CREATE EXTERNAL TABLE raw_weatherData (  
  location_id INT,  
  raw_date STRING,  
  weather_code INT,  
  temperature_2m_max DOUBLE,  
  temperature_2m_min DOUBLE,  
  temperature_2m_mean DOUBLE,  
  apparent_temperature_max DOUBLE,  
  apparent_temperature_min DOUBLE,  
  apparent_temperature_mean DOUBLE,  
  daylight_duration DOUBLE,  
  sunshine_duration DOUBLE,  
  precipitation_sum DOUBLE,  
  rain_sum DOUBLE,  
  precipitation_hours INT,  
  wind_speed_10m_max DOUBLE,  
  wind_gusts_10m_max DOUBLE,  
  wind_direction_10m_dominant INT,  
  shortwave_radiation_sum DOUBLE,  
  et0_fao_evapotranspiration DOUBLE,  
  sunrise STRING,  
  sunset STRING  
)  
ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.OpenCSVSerde'  
WITH SERDEPROPERTIES (  
  "separatorChar" = ","  
)  
STORED AS TEXTFILE  
LOCATION '/data/raw/weather'  
TBLPROPERTIES ("skip.header.line.count"="1");
```

create weatherData table.sql

```
CREATE TABLE weatherData AS
SELECT
    location_id,
    TO_DATE(FROM_UNIXTIME(UNIX_TIMESTAMP(raw_date, 'M/d/yyyy')) AS processed_date,
    weather_code,
    temperature_2m_max,
    temperature_2m_min,
    temperature_2m_mean,
    apparent_temperature_max,
    apparent_temperature_min,
    apparent_temperature_mean,
    daylight_duration,
    sunshine_duration,
    precipitation_sum,
    rain_sum,
    precipitation_hours,
    wind_speed_10m_max,
    wind_gusts_10m_max,
    wind_direction_10m_dominant,
    shortwave_radiation_sum,
    et0_fao_evapotranspiration,
    sunrise,
    sunset
FROM raw_weatherData;
```

validate_data_tables.sql

```
SELECT *
FROM locationData
LIMIT 10;

SELECT COUNT(*) AS total_rows
FROM locationData;

SELECT *
FROM weatherData
LIMIT 10;

SELECT COUNT(*) AS total_rows
FROM weatherData;
```

Code Listings:

query top 10 most temperate cities.sql

```
SELECT
    l.city_name,
    SUM(w.temperature_2m_max) AS temperature_2m_max_sum
FROM weatherData w
JOIN locationData l
    ON w.location_id = l.location_id
GROUP BY l.city_name
ORDER BY temperature_2m_max_sum ASC
LIMIT 10;
```

Question 02

Outputs/Results:

```
hive> SELECT
>   l.city_name,
>   CASE
>     WHEN MONTH(w.processed_date) IN (9,10,11,12,1,2,3) THEN 'Sep-Mar'
>     ELSE 'Apr-Aug'
>   END AS season,
>   AVG(w.et0_fao_evapotranspiration) AS et0_fao_evapotranspiration_avg
> FROM weatherData w
> JOIN locationData l
> ON w.location_id = l.location_id
> GROUP BY
>   l.city_name,
>   CASE
>     WHEN MONTH(w.processed_date) IN (9,10,11,12,1,2,3) THEN 'Sep-Mar'
>     ELSE 'Apr-Aug'
>   END;
```

Total MapReduce CPU Time Spent: 0 msec

OK

Ampara	Apr-Aug	4.8740841248303965
Ampara	Sep-Mar	3.7999967341606746
Anuradhapura	Apr-Aug	4.757299864314797
Anuradhapura	Sep-Mar	3.8852057478772046
Badulla	Apr-Aug	4.40969244685663
Badulla	Sep-Mar	3.545774003919002
Bandarawela	Apr-Aug	3.91542288557214
Bandarawela	Sep-Mar	3.4297877204441467
Batticaloa	Apr-Aug	5.028701944821341
Batticaloa	Sep-Mar	3.924924885695618
Colombo	Apr-Aug	3.6391768430574363
Colombo	Sep-Mar	3.814346832135855
Galle	Apr-Aug	3.9203120759837167
Galle	Sep-Mar	3.865094709340305
Gampaha	Apr-Aug	3.594884667571234
Gampaha	Sep-Mar	3.8763063357282848
Hambantota	Apr-Aug	4.9473857982813145
Hambantota	Sep-Mar	4.371035271064656
Jaffna	Apr-Aug	5.204776119402992
Jaffna	Sep-Mar	4.3129065969954254
Kalutara	Apr-Aug	3.649045680687465
Kalutara	Sep-Mar	3.7605486610058803
Kandy	Apr-Aug	3.785463591135227
Kandy	Sep-Mar	3.665035924232516
Kegalle	Apr-Aug	3.6006919945725975
Kegalle	Sep-Mar	3.8169627694317465
Kilinochchi[1]	Apr-Aug	5.659194934418823
Kilinochchi[1]	Sep-Mar	4.261583932070535
Kurunegala	Apr-Aug	3.8438037087290766
Kurunegala	Sep-Mar	3.9346538210320166

Mannar	Apr-Aug	5.285413839891462
Mannar	Sep-Mar	4.384591770084905
Matale	Apr-Aug	3.9505563093622804
Matale	Sep-Mar	3.762416721097326
Matara	Apr-Aug	3.974690185436467
Matara	Sep-Mar	3.963579359895494
Moneragala	Apr-Aug	4.696951605608322
Moneragala	Sep-Mar	3.8398269105159937
Mullaitivu	Apr-Aug	5.6241474445952155
Mullaitivu	Sep-Mar	4.2674003919007255
Nuwara Eliya	Apr-Aug	3.2327137042062404
Nuwara Eliya	Sep-Mar	3.1491606792945825
Polonnaruwa	Apr-Aug	5.713184079601993
Polonnaruwa	Sep-Mar	4.0113716525146925
Puttalam	Apr-Aug	4.237937584803256
Puttalam	Sep-Mar	4.019314173742645
Ratnapura	Apr-Aug	3.3212392582541788
Ratnapura	Sep-Mar	3.5609046374918316
Trincomalee	Apr-Aug	5.687069199457263
Trincomalee	Sep-Mar	4.203638145003268
Vavuniya	Apr-Aug	5.179837177747631
Vavuniya	Sep-Mar	3.95704768125407
Welimada	Apr-Aug	4.114572591587516
Welimada	Sep-Mar	3.422034617896794

Time taken: 8.768 seconds, Fetched: 54 row(s)

Code Listings:

query average evapotranspiration for each major agricultural season in each district.sql

```
SELECT
    l.city_name,
    CASE
        WHEN MONTH(w.processed_date) IN (9,10,11,12,1,2,3) THEN 'Sep-Mar '
        ELSE 'Apr-Aug'
    END AS season,
    AVG(w.et0_fao_evapotranspiration) AS et0_fao_evapotranspiration_avg
FROM weatherData w
JOIN locationData l
    ON w.location_id = l.location_id
GROUP BY
    l.city_name,
    CASE
        WHEN MONTH(w.processed_date) IN (9,10,11,12,1,2,3) THEN 'Sep-Mar '
        ELSE 'Apr-Aug'
    END;
```

Section 03 - Spark Analysis

Shell Commands Used:

- `docker exec -it namenode /bin/bash`
- `docker exec -it spark-master /bin/bash`

- `hdfs dfs -mkdir -p /data/raw`

- `hdfs dfs -put /home/data/raw/locationData.csv /data/raw`
- `hdfs dfs -put /home/data/raw/weatherData.csv /data/raw`

- `hdfs dfs -ls /data/raw`

- `/spark/bin/spark-sql`

Question 01

Outputs/Results:

```
spark-sql> CREATE OR REPLACE TEMPORARY VIEW weather_with_month AS
> SELECT
>   *,
>   YEAR(processed_date) AS year,
>   MONTH(processed_date) AS month
> FROM weatherData;
```

```
spark-sql>
> SELECT
>   year,
>   month,
>   SUM(CASE WHEN shortwave_radiation_sum > 15 THEN shortwave_radiation_sum ELSE 0
END) * 100.0
>   / SUM(shortwave_radiation_sum) AS percentage_above_15
> FROM weather_with_month
> GROUP BY year, month
> ORDER BY year, month;
```

2010	1	90.6664327754854
2010	2	92.5006553960077
2010	3	99.64173483996984
2010	4	97.22601380487558
2010	5	89.1514132996921
2010	6	89.50007405025544
2010	7	89.50856441777009
2010	8	85.92729157116374
2010	9	83.89490821844772
2010	10	86.57159035660716
2010	11	66.21163537930883
2010	12	56.78683119111537
2011	1	60.53587929534364
2011	2	83.6077398643956
2011	3	97.87309119129824
2011	4	91.94483593318145
2011	5	98.56547561083381
2011	6	97.04229623096386
2011	7	96.27882537289622
2011	8	94.36814665009183
2011	9	95.58417288579899
2011	10	86.6027870039275
2011	11	72.06521272569057
2011	12	71.2270350790081
2012	1	94.42675328160149
2012	2	90.15639466206942
2012	3	99.85923240962488
2012	4	95.8661750614765
2012	5	98.53247391767114
2012	6	94.89627262240211
2012	7	96.34072186036656

2012	8	96.82149111233718
2012	9	94.86098359842187
2012	10	79.78312748883899
2012	11	78.3327295156771
2012	12	60.418421803240086
2013	1	76.61872799018174
2013	2	88.99464451293862
2013	3	98.47983536679735
2013	4	99.01530695418033
2013	5	90.15856525135005
2013	6	83.13728243545907
2013	7	92.74332885390672
2013	8	97.35530685660756
2013	9	88.09811213952523
2013	10	92.21524204426237
2013	11	89.24840162897438
2013	12	62.92057924252644
2014	1	80.84078399755795
2014	2	99.63964085534234
2014	3	98.64320799617239
2014	4	97.73371827018623
2014	5	95.06196988513932
2014	6	95.8555872724126
2014	7	96.72487331720873
2014	8	93.85960335974123
2014	9	98.14923608484763
2014	10	84.7238862792742
2014	11	64.06904787753774
2014	12	57.02842120682722
2015	1	94.00759660159362
2015	2	92.54525889138203
2015	3	97.8592950702246
2015	4	96.88965091977178
2015	5	94.00481961113884
2015	6	91.6888896491905
2015	7	93.83885830540929
2015	8	97.62824964696624
2015	9	89.458895948504
2015	10	81.52520138291713
2015	11	68.16120586125975
2015	12	61.780771726818394
2016	1	91.59735805886528
2016	2	96.58095416979715
2016	3	100.0
2016	4	100.0
2016	5	81.51281339467543
2016	6	93.59170002386851
2016	7	92.09203824175557
2016	8	98.66184164801754
2016	9	98.13868227785728
2016	10	97.60051311401924
2016	11	71.51574994851394
2016	12	90.18624972488345
2017	1	84.89912803655186

2017	2	93.28871393923227
2017	3	97.14549480542938
2017	4	99.39205066582767
2017	5	91.14376795196748
2017	6	87.99951843340864
2017	7	95.35461384096513
2017	8	88.11241665258588
2017	9	88.0392123903623
2017	10	80.43404951622087
2017	11	59.6382089594222
2017	12	80.69777923342686
2018	1	88.998899961008
2018	2	94.37523164637815
2018	3	93.06921659794716
2018	4	97.96876227851054
2018	5	81.04119353957972
2018	6	91.90414276543108
2018	7	93.4621686841489
2018	8	93.24654977142177
2018	9	97.09403741534234
2018	10	71.1668841398884
2018	11	69.59760391457918
2018	12	80.10427573238745
2019	1	92.40507995576543
2019	2	93.86209978948905
2019	3	99.79241556359354
2019	4	93.12187255436714
2019	5	97.37079000643494
2019	6	92.87933981264503
2019	7	96.42578042939374
2019	8	79.67746856861507
2019	9	89.39279708878638
2019	10	70.31120163638612
2019	11	85.29043419421608
2019	12	62.27897684801354
2020	1	97.26118819979894
2020	2	99.0962894979909
2020	3	99.4510459840889
2020	4	99.7639893414541
2020	5	86.25840725698801
2020	6	90.15342605384205
2020	7	85.30579602085515
2020	8	90.75200575780289
2020	9	73.09792448730792
2020	10	97.29618627397859
2020	11	79.27416101983756
2020	12	72.58092803672088
2021	1	67.41711259963324
2021	2	97.49108306533113
2021	3	93.75901107686079
2021	4	98.71173699775606
2021	5	87.35636853300444
2021	6	91.5293319683912
2021	7	92.96755033771225

2021	8	87.85653875707146
2021	9	90.47479776221677
2021	10	82.40301076936369
2021	11	56.51940843639869
2021	12	89.66932843257793
2022	1	92.7615629153676
2022	2	95.54797509963637
2022	3	96.74291024145512
2022	4	95.81431919374155
2022	5	92.66660566688017
2022	6	92.99472042901914
2022	7	89.03946618450196
2022	8	89.85048063529587
2022	9	95.93255680600527
2022	10	79.96449157450563
2022	11	76.88367213172128
2022	12	76.89029827798281
2023	1	91.90518058364822
2023	2	92.71761042996928
2023	3	97.31399844521559
2023	4	98.01238686244538
2023	5	89.66582234299207
2023	6	97.15840375822832
2023	7	90.56397801149998
2023	8	99.65633301933723
2023	9	86.95807504328857
2023	10	83.97828372334637
2023	11	66.92220795319473
2023	12	66.3454344806138
2024	1	87.52004484439595
2024	2	99.66582975751841
2024	3	99.85312768216531
2024	4	98.69380560630816
2024	5	84.5899667213683
2024	6	91.43990061862397

Time taken: 3.1 seconds, Fetched 174 row(s)

25/12/17 21:13:04 INFO CliDriver: Time taken: 3.1 seconds, Fetched 174 row(s)

Code Listings: Preprocessing

create locationData table.sql

```
CREATE TABLE locationData
USING csv
OPTIONS (
  path "/data/raw/locationData.csv",
  header "true",
  inferSchema "true"
);
```

create weatherData temporary view.sql

```
CREATE OR REPLACE TEMPORARY VIEW weatherData_tmp_view
USING csv
OPTIONS (
  path "/data/raw/weatherData.csv",
  header "true",
  inferSchema "true"
);
```

create weatherData table.sql

```
CREATE TABLE weatherData AS
SELECT
    location_id,
    to_date(from_unixtime(unix_timestamp(date, 'M/d/yyyy'))) AS processed_date,
    `weather_code (wmo code)` AS weather_code,
    `temperature_2m_max (°C)` AS temperature_2m_max,
    `temperature_2m_min (°C)` AS temperature_2m_min,
    `temperature_2m_mean (°C)` AS temperature_2m_mean,
    `apparent_temperature_max (°C)` AS apparent_temperature_max,
    `apparent_temperature_min (°C)` AS apparent_temperature_min,
    `apparent_temperature_mean (°C)` AS apparent_temperature_mean,
    `daylight_duration (s)` AS daylight_duration,
    `sunshine_duration (s)` AS sunshine_duration,
    `precipitation_sum (mm)` AS precipitation_sum,
    `rain_sum (mm)` AS rain_sum,
    `precipitation_hours (h)` AS precipitation_hours,
    `wind_speed_10m_max (km/h)` AS wind_speed_10m_max,
    `wind_gusts_10m_max (km/h)` AS wind_gusts_10m_max,
    `wind_direction_10m_dominant (°)` AS wind_direction_10m_dominant,
    `shortwave_radiation_sum (MJ/m²)` AS shortwave_radiation_sum,
    `et0_fao_evapotranspiration (mm)` AS et0_fao_evapotranspiration,
    sunrise,
    sunset
FROM weatherData_tmp_view;
```

validate_data_tables.sql

```
SELECT *
FROM locationData
LIMIT 10;

SELECT COUNT(*) AS total_rows
FROM locationData;

SELECT *
FROM weatherData
LIMIT 10;

SELECT COUNT(*) AS total_rows
FROM weatherData;
```

Code Listings:

query monthly shortwave radiation percentage over 15MJ.sql

```
CREATE OR REPLACE TEMPORARY VIEW weather_with_month AS
SELECT
    *,
    YEAR(processed_date) AS year,
    MONTH(processed_date) AS month
FROM weatherData;

SELECT
    year,
    month,
    SUM(CASE WHEN shortwave_radiation_sum > 15 THEN shortwave_radiation_sum ELSE 0 END) *
100.0
    / SUM(shortwave_radiation_sum) AS percentage_above_15
FROM weather_with_month
GROUP BY year, month
ORDER BY year, month;
```

Question 02

Outputs/Results:

```
spark-sql> WITH monthly_avg AS (  
  > SELECT  
  >   year(processed_date) AS year,  
  >   month(processed_date) AS month,  
  >   AVG(temperature_2m_max) AS temperature_2m_max_avg  
  > FROM weatherData  
  > GROUP BY year(processed_date), month(processed_date)  
  > ),  
  >  
  > hottest_months AS (  
  > SELECT year, month  
  > FROM (  
  >   SELECT *,  
  >   MAX(temperature_2m_max_avg) OVER (PARTITION BY year) AS max_temp  
  >   FROM monthly_avg  
  > ) AS hottest_months_with_max_temp  
  > WHERE temperature_2m_max_avg = max_temp  
  > )  
  >  
  > SELECT  
  >   h.year,  
  >   h.month,  
  >   weekofyear(w.processed_date) AS week,  
  >   MAX(w.temperature_2m_max) AS temperature_2m_max_weekly_max  
  > FROM weatherData w  
  > JOIN hottest_months h  
  > ON year(w.processed_date) = h.year  
  > AND month(w.processed_date) = h.month  
  > GROUP BY  
  >   h.year,  
  >   h.month,  
  >   weekofyear(w.processed_date)  
  > ORDER BY  
  >   h.year,  
  >   h.month,  
  >   week;
```

2010	3	9	36.1
2010	3	10	36.7
2010	3	11	37.2
2010	3	12	36.6
2010	3	13	33.9
2011	6	22	35.1
2011	6	23	35.2
2011	6	24	35.5
2011	6	25	36.4
2011	6	26	37.0
2012	5	18	35.4
2012	5	19	37.1
2012	5	20	37.4

2012	5	21	36.2
2012	5	22	36.4
2013	4	14	36.5
2013	4	15	36.0
2013	4	16	36.2
2013	4	17	34.8
2013	4	18	34.8
2014	4	14	37.2
2014	4	15	35.2
2014	4	16	35.6
2014	4	17	35.1
2014	4	18	34.6
2015	7	27	35.1
2015	7	28	35.5
2015	7	29	36.0
2015	7	30	34.3
2015	7	31	36.4
2016	4	13	36.0
2016	4	14	35.5
2016	4	15	36.4
2016	4	16	37.8
2016	4	17	38.0
2017	4	13	35.3
2017	4	14	37.0
2017	4	15	38.4
2017	4	16	38.8
2017	4	17	39.3
2018	4	13	35.9
2018	4	14	36.4
2018	4	15	33.9
2018	4	16	36.4
2018	4	17	37.9
2018	4	18	37.5
2019	5	18	37.7
2019	5	19	38.8
2019	5	20	38.3
2019	5	21	37.4
2019	5	22	38.3
2020	3	9	36.0
2020	3	10	36.7
2020	3	11	37.0
2020	3	12	36.6
2020	3	13	36.7
2020	3	14	35.8
2021	4	13	37.8
2021	4	14	36.4
2021	4	15	33.2
2021	4	16	34.5
2021	4	17	34.9
2022	3	9	34.9
2022	3	10	35.5
2022	3	11	34.2
2022	3	12	35.0
2022	3	13	35.2

2023	8	31	39.5
2023	8	32	38.4
2023	8	33	39.3
2023	8	34	39.4
2023	8	35	37.6
2024	4	14	39.4
2024	4	15	38.9
2024	4	16	39.0
2024	4	17	39.4
2024	4	18	38.4

Time taken: 3.098 seconds, Fetched 77 row(s)

25/12/17 21:50:11 INFO CliDriver: Time taken: 3.098 seconds, Fetched 77 row(s)

Code Listings:

query weekly maximum temperatures for hottest months.sql

```
WITH monthly_avg AS (  
    SELECT  
        year(processed_date) AS year,  
        month(processed_date) AS month,  
        AVG(temperature_2m_max) AS temperature_2m_max_avg  
    FROM weatherData  
    GROUP BY year(processed_date), month(processed_date)  
),  
  
hottest_months AS (  
    SELECT year, month  
    FROM (  
        SELECT *,  
            MAX(temperature_2m_max_avg) OVER (PARTITION BY year) AS max_temp  
        FROM monthly_avg  
    ) AS hottest_months_with_max_temp  
    WHERE temperature_2m_max_avg = max_temp  
)  
  
SELECT  
    h.year,  
    h.month,  
    weekofyear(w.processed_date) AS week,  
    MAX(w.temperature_2m_max) AS temperature_2m_max_weekly_max  
FROM weatherData w  
JOIN hottest_months h  
ON year(w.processed_date) = h.year  
AND month(w.processed_date) = h.month  
GROUP BY  
    h.year,  
    h.month,  
    weekofyear(w.processed_date)  
ORDER BY  
    h.year,  
    h.month,  
    week;
```

- Task 3 -

Evapotranspiration Analysis using SparkMLlib

1. Verifying Data Availability in the Container

```
%spark.pyspark

!ls /home/data/raw
```

locationData.csv weatherData.csv

2. Importing Required Libraries and Configuring the Environment

```
%spark.pyspark

import math
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd

from itertools import product
from pyspark.ml.evaluation import RegressionEvaluator
from pyspark.ml.feature import StandardScaler, VectorAssembler
from pyspark.ml.regression import GBTRegressor, GeneralizedLinearRegression,
RandomForestRegressor
from pyspark.ml.tuning import CrossValidator, ParamGridBuilder
from pyspark.sql import Row
from pyspark.sql import functions
from pyspark.sql.functions import col, mean, month, round, sum, to_date, when
from pyspark.sql.window import Window

from pyspark.sql.functions import max as pyspark_sql_functions_max
from pyspark.sql.functions import min as pyspark_sql_functions_min

pd.set_option("display.max_columns", None)
pd.set_option('display.max_colwidth', None)
```

3. Loading Raw Data into Spark DataFrames and Inspecting Schemas

```
%spark.pyspark

location = spark.read \
    .option("header", "true") \
    .option("inferSchema", "true") \
    .csv("/home/data/raw/locationData.csv")

weather = spark.read \
```

```
.option("header", "true") \  
.option("inferSchema", "true") \  
.csv("/home/data/raw/weatherData.csv")
```

```
%spark.pyspark
```

```
location.printSchema()
```

root

```
|-- location_id: integer (nullable = true)  
|-- latitude: double (nullable = true)  
|-- longitude: double (nullable = true)  
|-- elevation: integer (nullable = true)  
|-- utc_offset_seconds: integer (nullable = true)  
|-- timezone: string (nullable = true)  
|-- timezone_abbreviation: integer (nullable = true)  
|-- city_name: string (nullable = true)
```

```
%spark.pyspark
```

```
weather.printSchema()
```

root

```
|-- location_id: integer (nullable = true)  
|-- date: string (nullable = true)  
|-- weather_code (wmo code): integer (nullable = true)  
|-- temperature_2m_max (°C): double (nullable = true)  
|-- temperature_2m_min (°C): double (nullable = true)  
|-- temperature_2m_mean (°C): double (nullable = true)  
|-- apparent_temperature_max (°C): double (nullable = true)  
|-- apparent_temperature_min (°C): double (nullable = true)  
|-- apparent_temperature_mean (°C): double (nullable = true)  
|-- daylight_duration (s): double (nullable = true)  
|-- sunshine_duration (s): double (nullable = true)  
|-- precipitation_sum (mm): double (nullable = true)  
|-- rain_sum (mm): double (nullable = true)  
|-- precipitation_hours (h): integer (nullable = true)  
|-- wind_speed_10m_max (km/h): double (nullable = true)  
|-- wind_gusts_10m_max (km/h): double (nullable = true)
```

```

|-- wind_direction_10m_dominant (°): integer (nullable = true)
|-- shortwave_radiation_sum (MJ/m²): double (nullable = true)
|-- et0_fao_evapotranspiration (mm): double (nullable = true)
|-- sunrise: string (nullable = true)
|-- sunset: string (nullable = true)

```

4. Joining Weather and Location Data and Checking for Missing Values

This step merges the weather and location DataFrames on the `location_id` column to create a combined dataset. It then calculates the number of missing values in each column to identify any nulls that may need to be handled before modeling.

```

%spark.pyspark

raw_data = weather.join(location, "location_id")

```

```

%spark.pyspark

null_counts = raw_data.select([
    sum(when(col(c).isNull(), 1).otherwise(0)).alias(c)
    for c in raw_data.columns
])

null_counts.toPandas()

```

location_id	date	weather_code (wmo code)	temperature_2m_max (°C)	temperature_2m_min (°C)	temperature_2m_mean (°C)	apparent_temperature_max (°C)	apparent_temperature_min (°C)	apparent_temperature_mean (°C)	daylight_duration (s)	sunshine_duration (s)	precipitation_sum (mm)	rain_sum (mm)	precipitation_hours (h)	wind_speed_10m_max (km/h)	wind_gusts_10m_max (km/h)	wind_direction_10m_dominant (°)	shortwave_radiation_sum (MJ/m²)	et0_fao_evapotranspiration (mm)	sunrise	sunset	latitude	longitude	elevation	utc_offset_seconds	timezone	timezone_abbreviation	city_name
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

5. Parsing Dates, Filtering for May, and Selecting Relevant Features

This step converts the date column to a proper date format, filters the dataset to include only records from the month of **May**, and selects the specific features (`precipitation_hours`, `sunshine_duration`, `wind_speed`) along with the target variable (`evapotranspiration`) for modeling.

```

%spark.pyspark

data_date_parsed = raw_data.withColumn("date_parsed", to_date(col("date"), "M/d/yyyy"))

```

```
%spark.pyspark
```

```
data_filtered_for_may = data_date_parsed.filter(month(col("date_parsed")) == 5)
```

```
%spark.pyspark
```

```
data_filtered_for_selected = data_filtered_for_may.select(  
    col("precipitation_hours (h)").alias("precipitation_hours"),  
    col("sunshine_duration (s)").alias("sunshine_duration"),  
    col("wind_speed_10m_max (km/h)").alias("wind_speed"),  
    col("et0_fao_evapotranspiration (mm)").alias("evapotranspiration")  
)
```

```
%spark.pyspark
```

```
data_filtered_for_selected.show()
```

precipitation_hours	sunshine_duration	wind_speed	evapotranspiration
14	40574.88	10.3	4.32
10	39600.0	9.2	4.29
9	39191.48	11.9	3.48
9	40046.57	13.4	3.76
9	40047.19	13.8	3.41
8	34909.82	11.3	3.58
10	38100.71	12.4	3.81
17	28622.78	11.0	3.27
18	5531.36	14.8	1.98
7	31854.57	14.3	3.26
11	39162.41	14.1	3.52
16	29706.92	11.3	2.96
18	33905.08	13.7	3.6
19	34314.11	14.0	2.69

6. Removing Duplicates and Aggregating by Feature Combinations

This step eliminates duplicate rows to ensure data uniqueness. It then groups the data by the selected input features (`precipitation_hours`, `sunshine_duration`, `wind_speed`) and calculates the **mean evapotranspiration for each unique combination**, reducing redundancy and preparing the dataset for regression modeling.

```
%spark.pyspark
```

```
data_filtered_for_selected.count()
```

12555

```
%spark.pyspark
```

```
data_dropped_duplicate_rows = data_filtered_for_selected.dropDuplicates()
```

```
%spark.pyspark
```

```
data_dropped_duplicate_rows.count()
```

12553

```
%spark.pyspark
```

```
data_selected_unique_features.count()
```

12484

7. Assembling Features into a Single Vector

This step uses Spark's `VectorAssembler` to combine the three input features (`precipitation_hours`, `sunshine_duration`, `wind_speed`) into a single feature vector column named `features`. This format is required by Spark ML regression models for training.

```
%spark.pyspark
```

```
assembler = VectorAssembler(  
    inputCols=["precipitation_hours", "sunshine_duration", "wind_speed"],  
    outputCol="features"  
)  
  
data_vector_assembled = assembler.transform(data_selected_unique_features)
```

```
%spark.pyspark
```

```
data_vector_assembled.show()
```

precipitation_hours	sunshine_duration	wind_speed	evapotranspiration	features
23	32791.46	23.5	3.96	[23.0, 32791.46, 23.5]
12	35029.06	14.7	3.18	[12.0, 35029.06, 14.7]
9	39447.63	8.1	3.91	[9.0, 39447.63, 8.1]
4	39671.39	8.4	4.68	[4.0, 39671.39, 8.4]
0	39910.75	45.2	6.04	[0.0, 39910.75, 45.2]
0	40869.39	18.7	4.17	[0.0, 40869.39, 18.7]
0	40759.13	21.9	5.87	[0.0, 40759.13, 21.9]
0	41057.92	32.8	5.64	[0.0, 41057.92, 32.8]
1	33101.19	13.6	4.4	[1.0, 33101.19, 13.6]
15	35785.96	15.0	3.47	[15.0, 35785.96, 15.0]
8	31929.46	16.9	4.02	[8.0, 31929.46, 16.9]
11	38220.86	14.0	4.19	[11.0, 38220.86, 14.0]
5	39964.88	9.3	4.22	[5.0, 39964.88, 9.3]
11	37519.83	18.1	2.99	[11.0, 37519.83, 18.1]

8. Scaling Features Using StandardScaler

This step applies StandardScaler to normalize the assembled feature vectors. By centering the data (subtracting the mean) and scaling to **unit variance**, it ensures that all features contribute equally during model training, improving convergence and performance for regression algorithms.

```
%spark.pyspark

scaler = StandardScaler(inputCol="features", outputCol="scaled_features", withMean=True,
withStd=True)

scaler_model = scaler.fit(data_vector_assembled)

data_scaled = scaler_model.transform(data_vector_assembled)
```

```
%spark.pyspark

data_scaled.show(truncate=False)
```

precipitation_hours	sunshine_duration	wind_speed	evapotranspiration	features	scaled_features
23	32791.46	23.5	3.96	[23.0, 32791.46, 23.5]	[2.162196050349384, -0.1667399179576334, 0.7299439901894567]
12	35029.06	14.7	3.18	[12.0, 35029.06, 14.7]	[0.5768819180010629, 0.07725770821034114, -0.41111100293814523]
9	39447.63	8.1	3.91	[9.0, 39447.63, 8.1]	[0.1445235182697027, 0.5590777792941041, -1.2669022477838465]
4	39671.39	8.4	4.68	[4.0, 39671.39, 8.4]	[-0.5760738146158977, 0.5834775419109017, -1.2280026457454056]
0	39910.75	45.2	6.04	[0.0, 39910.75, 45.2]	[-1.1525516809243779, 0.6095783963082769, 3.543681870970021]
0	40869.39	18.7	4.17	[0.0, 40869.39, 18.7]	[-1.1525516809243779, 0.7141126671116677, 0.10755035757440105]
0	40759.13	21.9	5.87	[0.0, 40759.13, 21.9]	[-1.1525516809243779, 0.7020894376420209, 0.522479445984438]
0	41057.92	32.8	5.64	[0.0, 41057.92, 32.8]	[-1.1525516809243779, 0.7346707974573272, 1.9358316533811264]
1	33101.19	13.6	4.4	[1.0, 33101.19, 13.6]	[-1.008432214347258, -0.13296561300902435, -0.5537428770790954]
15	35785.96	15.0	3.47	[15.0, 35785.96, 15.0]	[1.0092403177324232, 0.15979337287182685, -0.37221140089970417]
8	31929.46	16.9	4.02	[8.0, 31929.46, 16.9]	[4.040516925826212E-4, -0.2607361432690359, -0.12584725465624488]
11	38220.86	14.0	4.19	[11.0, 38220.86, 14.0]	[0.4327624514239429, 0.4253054482903118, -0.5018767410278407]
5	39964.88	9.3	4.22	[5.0, 39964.88, 9.3]	[-0.4319543480387776, 0.615480966698191, -1.1113038396300825]
11	37519.83	18.1	2.99	[11.0, 37519.83, 18.1]	[0.4327624514239429, 0.34886209117888217, 0.029751153497519386]

9. Preparing Final Processed Data for Modeling

This step renames the scaled feature column to features and the target column to label, creating a dataset in the format required by Spark ML regression models.

```
%spark.pyspark

processed_data = data_scaled.select(col("scaled_features").alias("features"),
col("evapotranspiration").alias("label"))
```

```
%spark.pyspark

print("Total processed data rows:", processed_data.count())
```

Total processed data rows: 12484

features	label
[2.162196050349384, -0.1667399179576334, 0.7299439901894567]	13.96
[0.5768819180010629, 0.07725770821034114, -0.41111100293814523]	13.18
[0.1445235182697027, 0.5590777792941041, -1.2669022477838465]	13.91
[-0.5760738146158977, 0.5834775419109017, -1.2280026457454056]	14.68
[-1.1525516809243779, 0.6095783963082769, 3.543681870970021]	16.04
[-1.1525516809243779, 0.7141126671116677, 0.10755035757440105]	14.17
[-1.1525516809243779, 0.7020894376420209, 0.522479445984438]	15.87
[-1.1525516809243779, 0.7346707974573272, 1.9358316533811264]	15.64
[-1.008432214347258, -0.13296561300902435, -0.5537428770790954]	14.4
[1.0092403177324232, 0.15979337287182685, -0.37221140089970417]	13.47
[4.040516925826212E-4, -0.2607361432690359, -0.12584725465624488]	14.02
[0.4327624514239429, 0.4253054482903118, -0.5018767410278407]	14.19
[-0.4319543480387776, 0.615480966698191, -1.1113038396300825]	14.22

10. Stratified Train-Test Split and Data Inspection

This step performs a stratified split of the processed dataset to create training and testing sets while preserving the distribution of the target variable (**evapotranspiration**).

- The label is divided into **1200 bins**, and **80%** of each bin is sampled for training.
- The remaining **20%** of each bin forms the test set.
- Histograms are plotted to visually compare the target distribution between train and test sets.
- Finally, summary statistics and row counts for both sets are displayed to verify the split and ensure consistent representation of all target ranges.

```
%spark.pyspark

num_bins = 1200

processed_data_binned = processed_data.withColumn(
    "label_bin",
    functions.ntile(num_bins).over(Window.orderBy("label"))
)

fractions = (processed_data_binned
    .groupBy("label_bin")
    .count()
    .select("label_bin", functions.lit(0.8).alias("fraction"))
    .rdd
    .map(lambda row: (row["label_bin"], row["fraction"]))
    .collectAsMap()
)

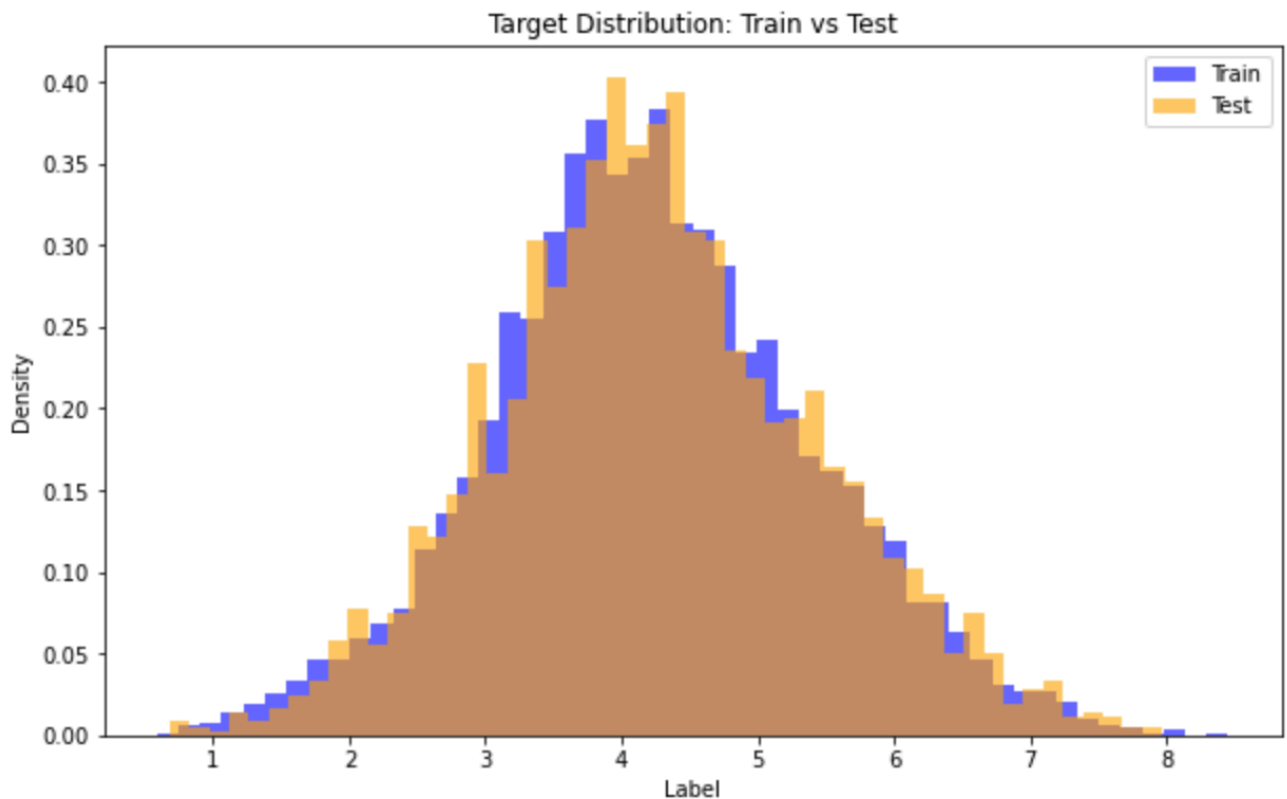
train_data_df = processed_data_binned.sampleBy("label_bin", fractions, seed=42)
test_data_df = processed_data_binned.subtract(train_data_df)

train_data = train_data_df.drop("label_bin")
test_data = test_data_df.drop("label_bin")
```

```
%spark.pyspark

train_labels = train_data.select("label").toPandas()
test_labels = test_data.select("label").toPandas()

# Plot histograms of train and test
plt.figure(figsize=(10,6))
plt.hist(train_labels['label'], bins=50, alpha=0.6, label='Train', color='blue',
density=True)
plt.hist(test_labels['label'], bins=50, alpha=0.6, label='Test', color='orange',
density=True)
plt.xlabel("Label")
plt.ylabel("Density")
plt.title("Target Distribution: Train vs Test")
plt.legend()
plt.show()
```



```
%spark.pyspark
```

```
print("Total rows in Train:", train_data.count())
print("Total rows in Test:", test_data.count())
```

Total rows in Train: 10013

Total rows in Test: 2471

```
%spark.pyspark
```

```
print("\nTrain statistics:")
train_data.select("label").describe().show()

print("\nTest statistics:")
test_data.select("label").describe().show()
```

Train statistics:

summary	label
count	10013
mean	4.222804936915348
stddev	1.1777183987959883
min	0.59
max	8.45

Test statistics:

summary	label
count	2471
mean	4.255259004451631
stddev	1.1777466191339454
min	0.68
max	7.97

```
%spark.pyspark

print("\nTrain counts per bin:")
train_data_df.groupBy("label_bin").count().orderBy("label_bin").show(n=num_bins)
```

Train counts per bin:

label_bin	count
1	8
2	9
3	11
4	10
5	8
6	9
7	10
8	9
9	11
10	10
11	11
12	10

```
%spark.pyspark

print("\nTest counts per bin:")
test_data_df.groupBy("label_bin").count().orderBy("label_bin").show(n=num_bins)
```

Test counts per bin:

label_bin	count
1	3
2	2
4	1
5	3
6	2
7	1
8	2
10	1
12	1
13	4
15	2
16	2

11. Caching Data and Setting Up the Evaluation Metric

This step caches the training and testing DataFrames in memory to speed up repeated access during model training and evaluation. It also initializes a RegressionEvaluator with **RMSE (Root Mean Squared Error)** as the metric to measure the performance of regression models.

```
%spark.pyspark  
  
train_data.cache()  
test_data.cache()
```

DataFrame[features: vector, label: double]

```
%spark.pyspark  
  
evaluator = RegressionEvaluator(  
    labelCol="label",  
    predictionCol="prediction",  
    metricName="rmse"  
)
```

12. Training Multiple Regression Models with Cross-Validation and Extracting Best Hyperparameters

This step performs model training and hyperparameter tuning for four regression algorithms using **5-fold cross-validation**:

- Generalized Linear Regression (Gaussian)** – trained with a log link function, tuned over multiple regParam values.
- Generalized Linear Regression (Gamma)** – also using a log link function with the same regParam grid.
- Random Forest Regressor** – tuned over the number of trees (`numTrees`) and maximum depth (`maxDepth`).
- Gradient Boosted Tree Regressor** – tuned over maximum iterations (`maxIter`) and tree depth (`maxDepth`).

After training:

- The best estimator from each cross-validated model is extracted.
- A custom function compiles a summary DataFrame showing each model's parameters, the grid of values used in cross-validation, and the selected best value.
- The final table displays only the parameters that were actually tuned, providing a clear overview of optimal hyperparameters for all models.

```
%spark.pyspark

glr_gaussian = GeneralizedLinearRegression(
    featuresCol="features",
    labelCol="label",
    family="gaussian",
    link="log"
)

glr_gaussian_param_grid = (
    ParamGridBuilder()
    .addGrid(glr_gaussian.regParam, [0.000001, 0.00001, 0.0001, 0.001, 0.01, 0.05, 0.1,
0.3])
    .build()
)

glr_gaussian_cv = CrossValidator(
    estimator=glr_gaussian,
    estimatorParamMaps=glr_gaussian_param_grid,
    evaluator=evaluator,
    numFolds=5
)

glr_gaussian_model = glr_gaussian_cv.fit(train_data)
```

```
%spark.pyspark

glr_gamma = GeneralizedLinearRegression(
    featuresCol="features",
    labelCol="label",
    family="gamma",
    link="log"
)

glr_gamma_param_grid = (
    ParamGridBuilder()
    .addGrid(glr_gaussian.regParam, [0.000001, 0.00001, 0.0001, 0.001, 0.01, 0.05, 0.1,
0.3])
    .build()
)

glr_gamma_cv = CrossValidator(
    estimator=glr_gamma,
    estimatorParamMaps=glr_gamma_param_grid,
    evaluator=evaluator,
    numFolds=5
)

glr_gamma_model = glr_gamma_cv.fit(train_data)
```

```
%spark.pyspark

rfr = RandomForestRegressor(
    featuresCol="features",
    labelCol="label",
    seed=42
)

rfr_param_grid = (
    ParamGridBuilder()
    .addGrid(rfr.numTrees, [50, 75, 100])
    .addGrid(rfr.maxDepth, [5, 10, 15])
    .build()
)

rfr_cv = CrossValidator(
    estimator=rfr,
    estimatorParamMaps=rfr_param_grid,
    evaluator=evaluator,
    numFolds=5
)

rfr_model = rfr_cv.fit(train_data)
```

```
%spark.pyspark

gbtr = GBTRegressor(
    featuresCol="features",
    labelCol="label",
    seed=42
)

gbtr_param_grid = (
    ParamGridBuilder()
    .addGrid(gbtr.maxIter, [150, 175, 200])
    .addGrid(gbtr.maxDepth, [1, 3, 5])
    .build()
)

gbtr_cv = CrossValidator(
    estimator=gbtr,
    estimatorParamMaps=gbtr_param_grid,
    evaluator=evaluator,
    numFolds=5
)

gbtr_model = gbtr_cv.fit(train_data)
```

```
%spark.pyspark

glr_gaussian_best_model_estimator = glr_gaussian_model.bestModel
glr_gamma_best_model_estimator = glr_gamma_model.bestModel
rfr_best_model_estimator = rfr_model.bestModel
gbtr_best_model_estimator = gbtr_model.bestModel

glr_gaussian_best_model_param_map = glr_gaussian_best_model_estimator.extractParamMap()
glr_gamma_best_model_param_map = glr_gamma_best_model_estimator.extractParamMap()
rfr_best_model_param_map = rfr_best_model_estimator.extractParamMap()
gbtr_best_model_param_map = gbtr_best_model_estimator.extractParamMap()
```

```
%spark.pyspark

def get_model_cv_params_df(
    cv_model,
    model_name,
    param_grid
):
    best_estimator = cv_model.bestModel

    best_param_map = {
        p.name: v for p, v in best_estimator.extractParamMap().items()
    }

    cv_params = {}
    for grid in param_grid:
        for p, v in grid.items():
            cv_params.setdefault(p.name, set()).add(v)

    cv_params = {
        k: sorted(list(v)) for k, v in cv_params.items()
    }

    rows = []
    for param in sorted(set(cv_params) | set(best_param_map)):
        rows.append({
            "model": model_name,
            "parameter": param,
            "cv_values": cv_params.get(param, None),
            "selected_value": best_param_map.get(param, None)
        })

    return pd.DataFrame(rows)
```

```
%spark.pyspark

glr_gaussian_params_df = get_model_cv_params_df(
    cv_model=glr_gaussian_model,
    model_name="GeneralizedLinearRegression (gaussian)",
    param_grid=glr_gaussian_param_grid
)

glr_gamma_params_df = get_model_cv_params_df(
    cv_model=glr_gamma_model,
    model_name="GeneralizedLinearRegression (gamma)",
    param_grid=glr_gamma_param_grid
)

rfr_params_df = get_model_cv_params_df(
    cv_model=rfr_model,
    model_name="RandomForestRegressor",
    param_grid=rfr_param_grid
)

gbtr_params_df = get_model_cv_params_df(
    cv_model=gbtr_model,
    model_name="GBTRegressor",
    param_grid=gbtr_param_grid
)

all_params_df = pd.concat([
    glr_gaussian_params_df,
    glr_gamma_params_df,
    rfr_params_df,
    gbtr_params_df
], ignore_index=True)
```

```
%spark.pyspark

all_params_df[all_params_df['cv_values'].notna()]
```

	model	parameter	cv_values	selected_value
7	GeneralizedLinearRegression (gaussian)	regParam	[1e-06, 1e-05, 0.0001, 0.001, 0.01, 0.05, 0.1, 0.3]	0.0001
18	GeneralizedLinearRegression (gamma)	regParam	[1e-06, 1e-05, 0.0001, 0.001, 0.01, 0.05, 0.1, 0.3]	0.0
29	RandomForestRegressor	maxDepth	[5, 10, 15]	10
33	RandomForestRegressor	numTrees	[50, 75, 100]	100
45	GBTRegressor	maxDepth	[1, 3, 5]	3
46	GBTRegressor	maxIter	[150, 175, 200]	200

13. Generating Predictions and Evaluating Model Performance

This step uses the trained models to predict **evapotranspiration** on both the training and test datasets. It evaluates each model using three metrics:

- **RMSE (Root Mean Squared Error)** – measures overall prediction error magnitude.
- **MAE (Mean Absolute Error)** – measures average absolute error.
- **R² (Coefficient of Determination)** – indicates how well the model explains variance in the target.

A helper function organizes the results into a structured DataFrame, allowing easy comparison of model performance across training and testing data.

```
%spark.pyspark

glr_gaussian_model_predictions_train_data = glr_gaussian_model.transform(train_data)
glr_gaussian_model_predictions_test_data = glr_gaussian_model.transform(test_data)

glr_gamma_model_predictions_train_data = glr_gamma_model.transform(train_data)
glr_gamma_model_predictions_test_data = glr_gamma_model.transform(test_data)

rfr_model_predictions_train_data = rfr_model.transform(train_data)
rfr_model_predictions_test_data = rfr_model.transform(test_data)

gbtr_model_predictions_train_data = gbtr_model.transform(train_data)
gbtr_model_predictions_test_data = gbtr_model.transform(test_data)
```

```
%spark.pyspark

evaluator_rmse = RegressionEvaluator(
    labelCol="label",
    predictionCol="prediction",
    metricName="rmse"
)

evaluator_mae = RegressionEvaluator(
    labelCol="label",
    predictionCol="prediction",
    metricName="mae"
)

evaluator_r2 = RegressionEvaluator(
    labelCol="label",
    predictionCol="prediction",
    metricName="r2"
)
```

```
def evaluate_model(predictions, model_name, data_type):
    return Row(
        Model=model_name,
        Data=data_type,
        RMSE=evaluator_rmse.evaluate(predictions),
        MAE=evaluator_mae.evaluate(predictions),
        R2=evaluator_r2.evaluate(predictions)
    )
```

```
%spark.pyspark
```

```
results = []
```

```
results.append(evaluate_model(glr_gaussian_model_predictions_train_data,
"GeneralizedLinearRegression (gaussian)", "Train"))
results.append(evaluate_model(glr_gaussian_model_predictions_test_data,
"GeneralizedLinearRegression (gaussian)", "Test"))
results.append(evaluate_model(glr_gamma_model_predictions_train_data,
"GeneralizedLinearRegression (gamma)", "Train"))
results.append(evaluate_model(glr_gamma_model_predictions_test_data,
"GeneralizedLinearRegression (gamma)", "Test"))
results.append(evaluate_model(rfr_model_predictions_train_data, "RandomForestRegressor",
"Train"))
results.append(evaluate_model(rfr_model_predictions_test_data, "RandomForestRegressor",
"Test"))
results.append(evaluate_model(gbtr_model_predictions_train_data, "GBTRegressor", "Train"))
results.append(evaluate_model(gbtr_model_predictions_test_data, "GBTRegressor", "Test"))
```

```
%spark.pyspark
```

```
results_df = spark.createDataFrame(results)
results_df_pd = (results_df.select("Model", "Data", "MAE", "RMSE", "R2").toPandas())
results_df.select("Model", "Data", "RMSE", "MAE", "R2").show(truncate=False)
```

Model	Data	RMSE	MAE	R2
GeneralizedLinearRegression (gaussian)	Train	0.5060866602244035	0.39296629679135525	0.8153240951367435
GeneralizedLinearRegression (gaussian)	Test	0.5074611734406371	0.39787552403202325	0.8142718646950949
GeneralizedLinearRegression (gamma)	Train	0.5077436256554333	0.3927745104612301	0.8141128301483752
GeneralizedLinearRegression (gamma)	Test	0.5095254663661855	0.3982388793639631	0.8127577504983252
RandomForestRegressor	Train	0.4472412655660615	0.3442282297077279	0.8557737900564671
RandomForestRegressor	Test	0.47415232876665153	0.36709979031220974	0.8378534041059369
GBTRegressor	Train	0.4620310212452926	0.35650399418372186	0.846077280944026
GBTRegressor	Test	0.4711795141723251	0.36405966925058997	0.8398802660623481

14. Visualizing Model Performance Metrics

This step creates bar plots to compare the performance of all trained models across the training and test datasets. Separate plots are generated for **RMSE**, **MAE**, and **R²**, allowing a

clear visual assessment of how each model performs and whether any overfitting or underfitting is present.

```
%spark.pyspark

metrics_pd = (
    results_df
    .select("Model", "Data", "MAE", "RMSE", "R2")
    .toPandas()
)

metrics_pd["Data"] = metrics_pd["Data"].replace({
    "train_data": "Train",
    "test_data": "Test"
})

def plot_metric_bar(metric_name):
    fig, ax = plt.subplots(figsize=(10, 6))

    models = metrics_pd["Model"].unique()
    x = np.arange(len(models))
    width = 0.35

    train_vals = (
        metrics_pd[metrics_pd["Data"] == "Train"]
        .set_index("Model")[metric_name]
        .loc[models]
    )

    test_vals = (
        metrics_pd[metrics_pd["Data"] == "Test"]
        .set_index("Model")[metric_name]
        .loc[models]
    )

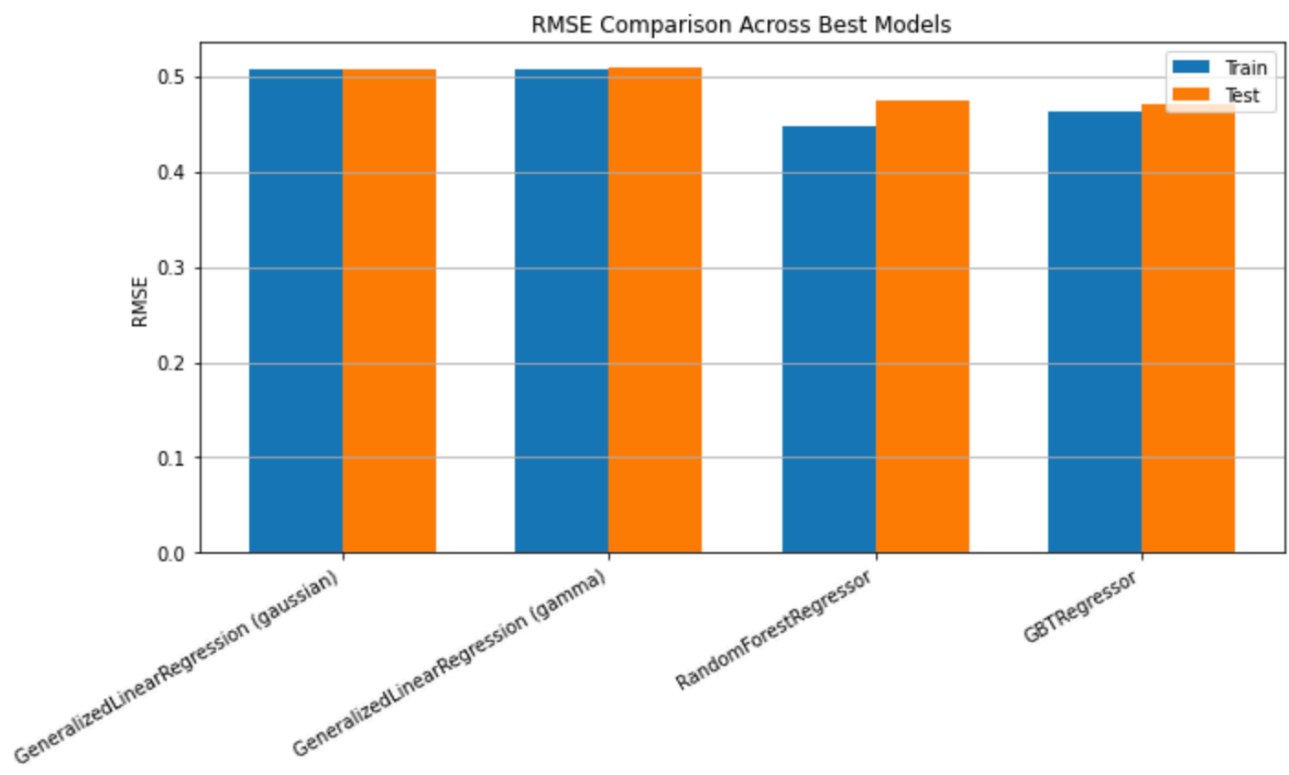
    ax.bar(x - width/2, train_vals, width, label="Train")
    ax.bar(x + width/2, test_vals, width, label="Test")

    ax.set_xticks(x)
    ax.set_xticklabels(models, rotation=30, ha="right")
    ax.set_ylabel(metric_name)
    ax.set_title(f"{metric_name} Comparison Across Best Models")
    ax.legend()
    ax.grid(axis="y")

plt.tight_layout()
plt.show()
```

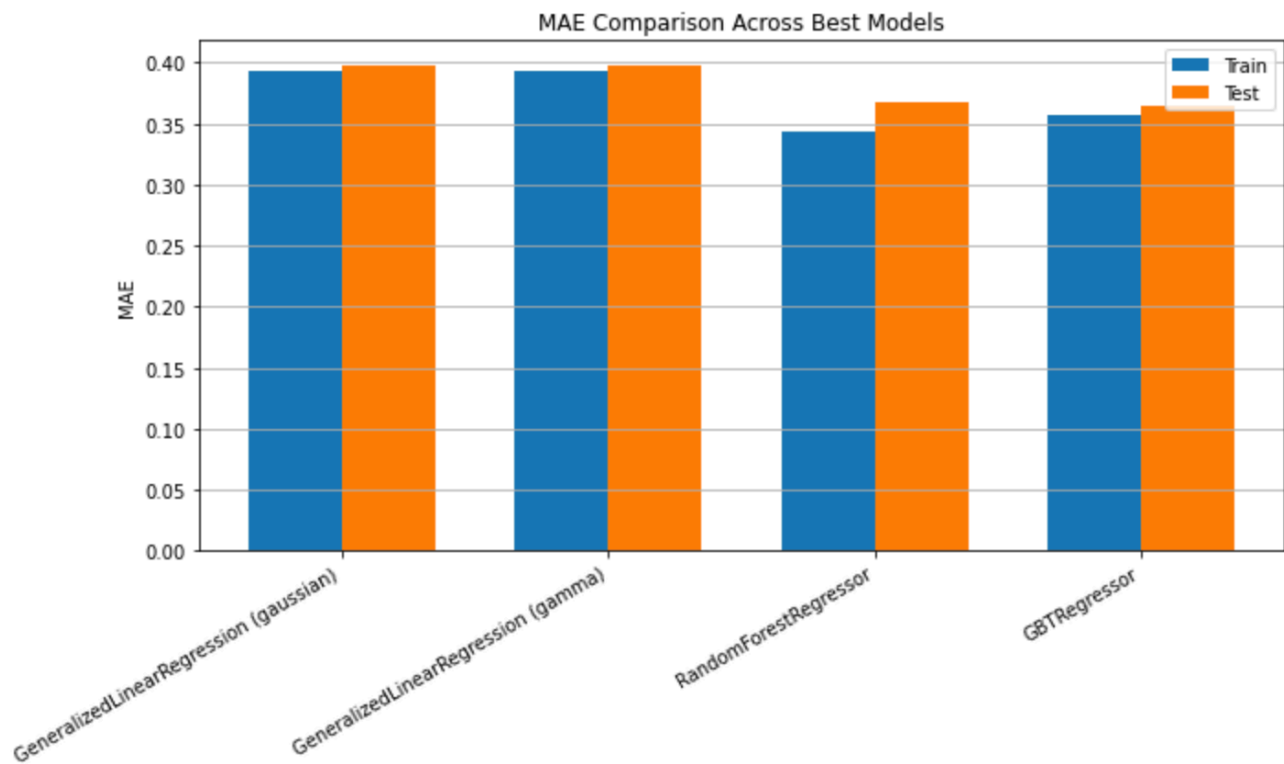
```
%spark.pyspark
%matplotlib inline

plot_metric_bar("RMSE")
```



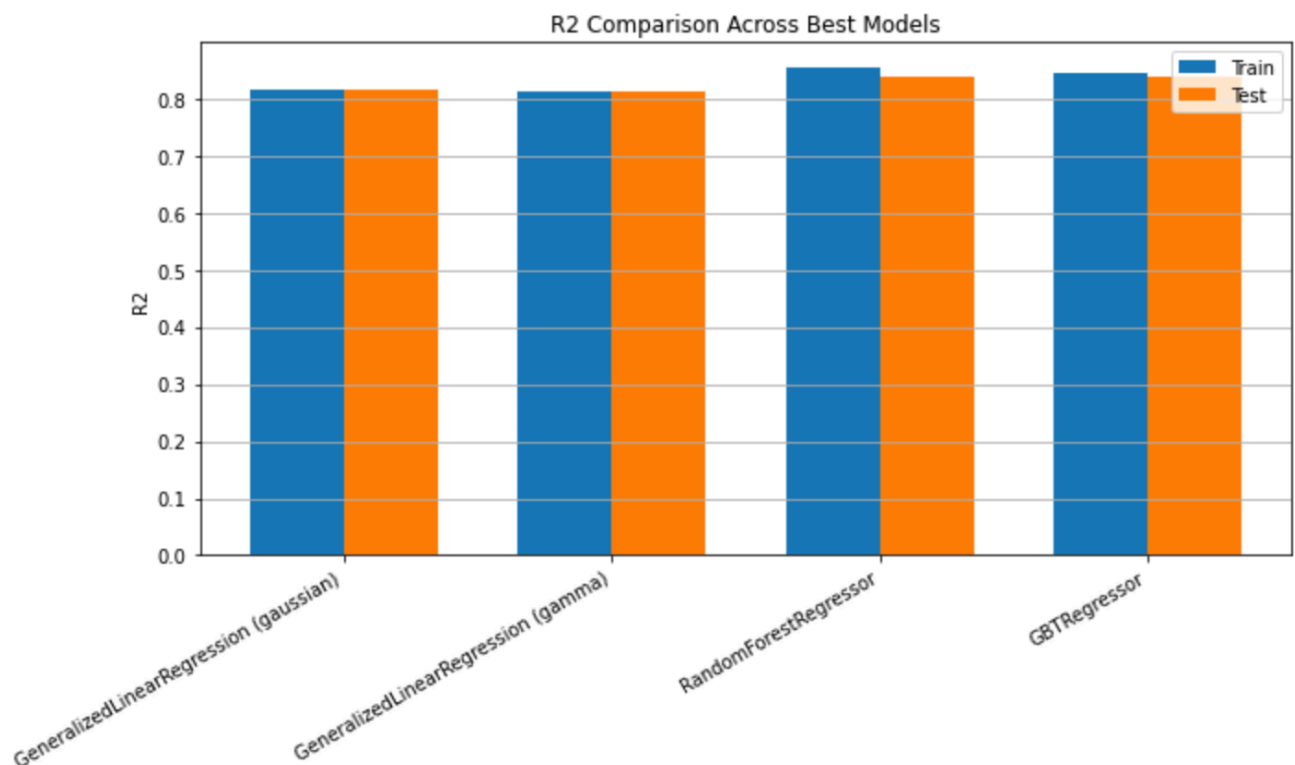
```
%spark.pyspark
%matplotlib inline

plot_metric_bar("MAE")
```



```
%spark.pyspark
%matplotlib inline

plot_metric_bar("R2")
```



15. Plotting Predicted vs. Actual Values

This step generates scatter plots comparing the predicted evapotranspiration values against the actual values for both training and test datasets. A **diagonal line** is added to each plot as a reference for perfect predictions, helping visually assess the accuracy and bias of each model.

```
%spark.pyspark

def to_pandas(df, label_col="label", pred_col="prediction", sample_frac=1.0):
    sdf = df.select(label_col, pred_col)
    if sample_frac < 1.0:
        sdf = sdf.sample(fraction=sample_frac, seed=42)
    return sdf.toPandas()

def plot_predicted_vs_actual(train_df, test_df, model_name):
    train_pd = to_pandas(train_df)
    test_pd = to_pandas(test_df)

    fig, axes = plt.subplots(1, 2, figsize=(14, 6))
```

```

for df, title, ax in [
    (train_pd, "Train Data", axes[0]),
    (test_pd, "Test Data", axes[1])
]:
    ax.scatter(df["label"], df["prediction"], alpha=0.5)

    min_val = min([df["label"].min(), df["prediction"].min()])
    max_val = max([df["label"].max(), df["prediction"].max()])
    ax.plot([min_val, max_val], [min_val, max_val], linestyle="--")

    ax.set_title(title)
    ax.set_xlabel("Actual")
    ax.set_ylabel("Predicted")
    ax.grid(True)

fig.suptitle(f"Predicted vs Actual - {model_name}", fontsize=14)
plt.tight_layout()
plt.show()

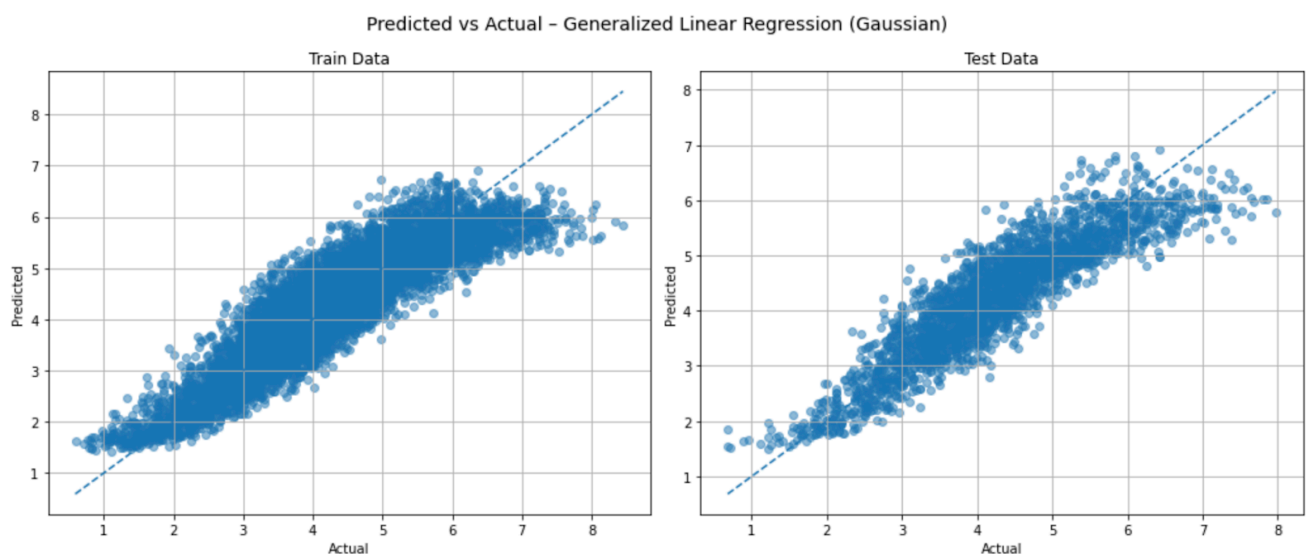
```

```

%spark.pyspark
%matplotlib inline

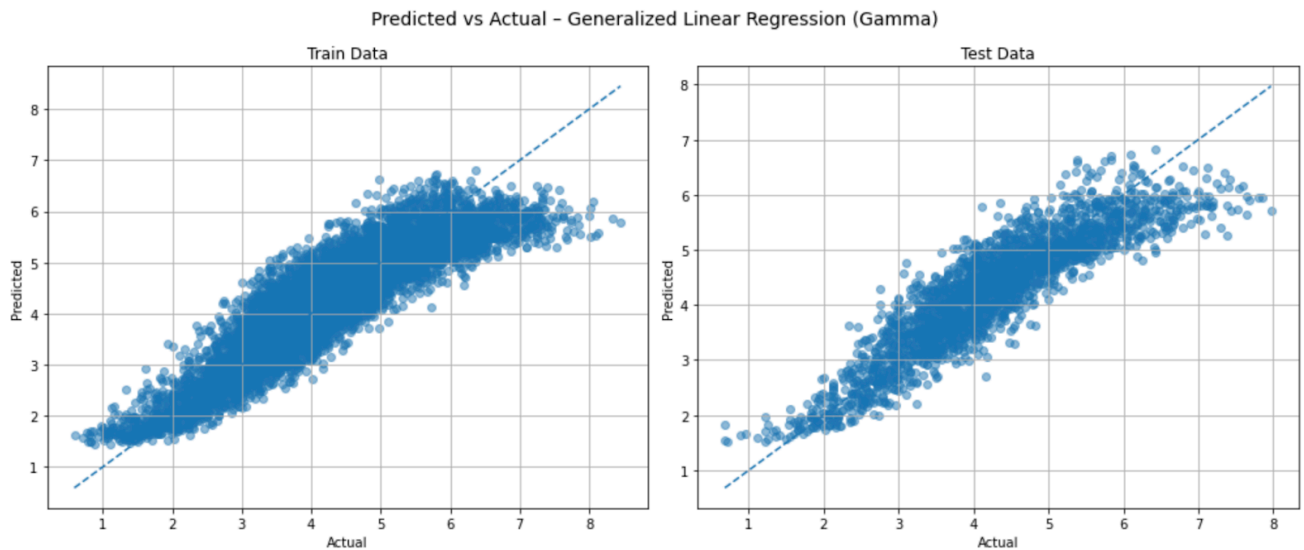
plot_predicted_vs_actual(
    glr_gaussian_model_predictions_train_data,
    glr_gaussian_model_predictions_test_data,
    "Generalized Linear Regression (Gaussian)"
)

```



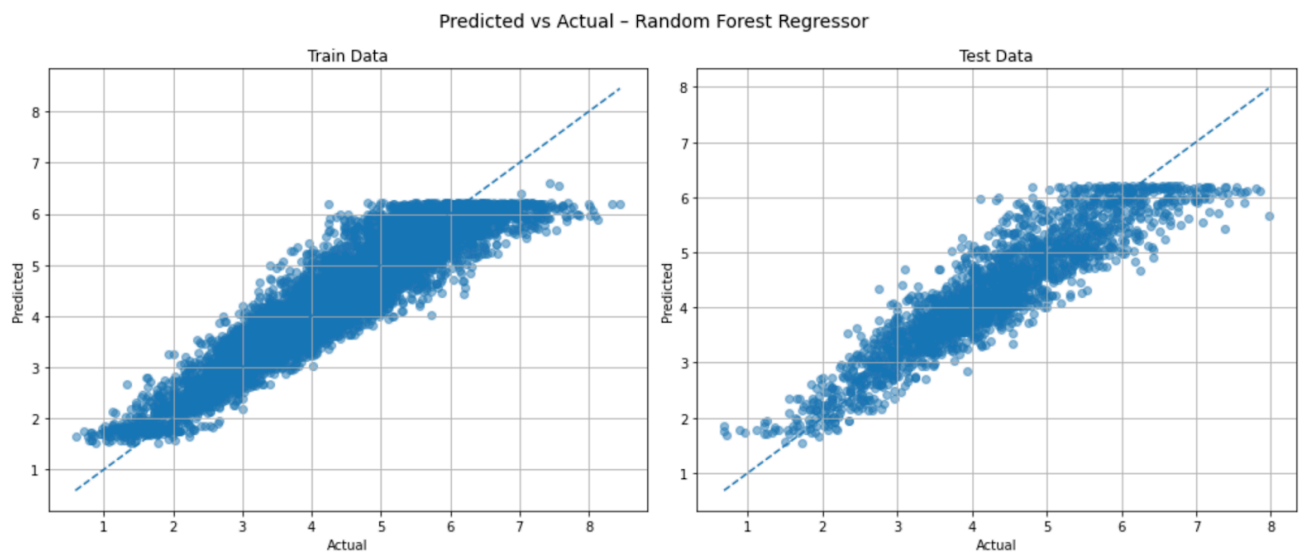
```
%spark.pyspark
%matplotlib inline

plot_predicted_vs_actual(
    glr_gamma_model_predictions_train_data,
    glr_gamma_model_predictions_test_data,
    "Generalized Linear Regression (Gamma)"
)
```



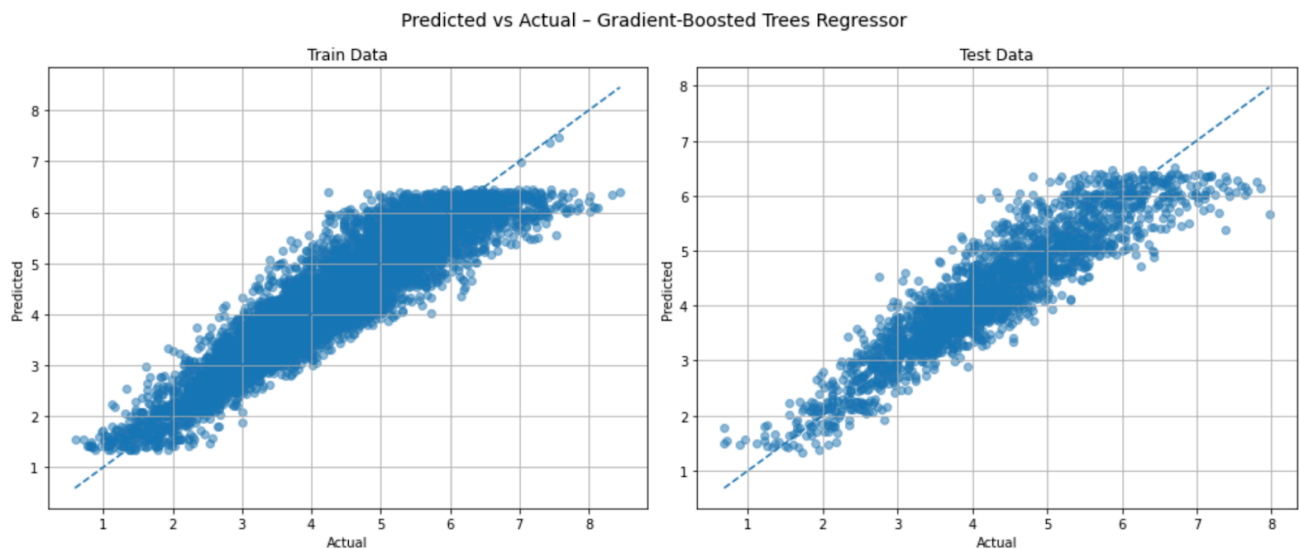
```
%spark.pyspark
%matplotlib inline

plot_predicted_vs_actual(
    rfr_model_predictions_train_data,
    rfr_model_predictions_test_data,
    "Random Forest Regressor"
)
```



```
%spark.pyspark
%matplotlib inline

plot_predicted_vs_actual(
    gbtr_model_predictions_train_data,
    gbtr_model_predictions_test_data,
    "Gradient-Boosted Trees Regressor"
)
```



16. Plotting Residuals vs. Actual Values

This step creates residual plots for each model, showing the difference between **actual** and **predicted evapotranspiration** values on both training and test datasets.

- The residuals help identify patterns, biases, or heteroscedasticity in model predictions.
- A horizontal line at zero is added as a reference for perfect predictions.

```
%spark.pyspark

def get_residual_df(df):
    return (
        df.selectExpr(
            "label as actual",
            "label - prediction as residual"
        )
        .toPandas()
    )

def plot_residuals_vs_actual(train_df, test_df, model_name):
    train_pd = get_residual_df(train_df)
    test_pd = get_residual_df(test_df)

    fig, axes = plt.subplots(1, 2, figsize=(14, 6))
```

```

# Train
axes[0].scatter(train_pd["actual"], train_pd["residual"], alpha=0.5)
axes[0].axhline(0, linestyle="--")
axes[0].set_title("Train Data")
axes[0].set_xlabel("Actual")
axes[0].set_ylabel("Residual (Actual - Predicted)")
axes[0].grid(True)

# Test
axes[1].scatter(test_pd["actual"], test_pd["residual"], alpha=0.5)
axes[1].axhline(0, linestyle="--")
axes[1].set_title("Test Data")
axes[1].set_xlabel("Actual")
axes[1].set_ylabel("Residual (Actual - Predicted)")
axes[1].grid(True)

fig.suptitle(f"Residual Plot - {model_name}", fontsize=14)
plt.tight_layout()
plt.show()

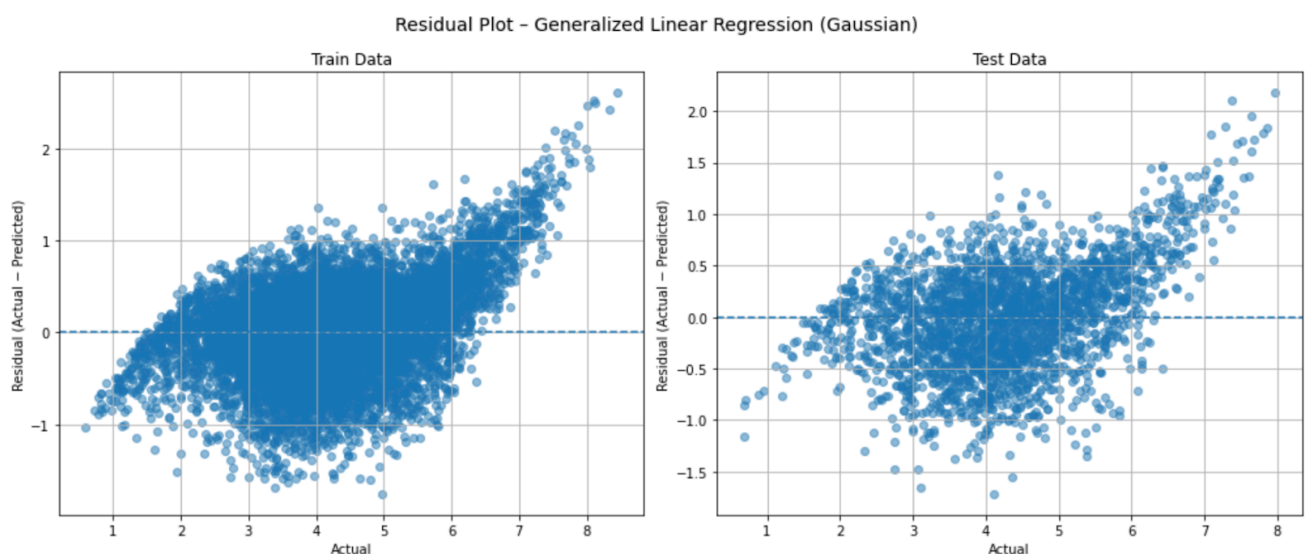
```

```

%spark.pyspark
%matplotlib inline

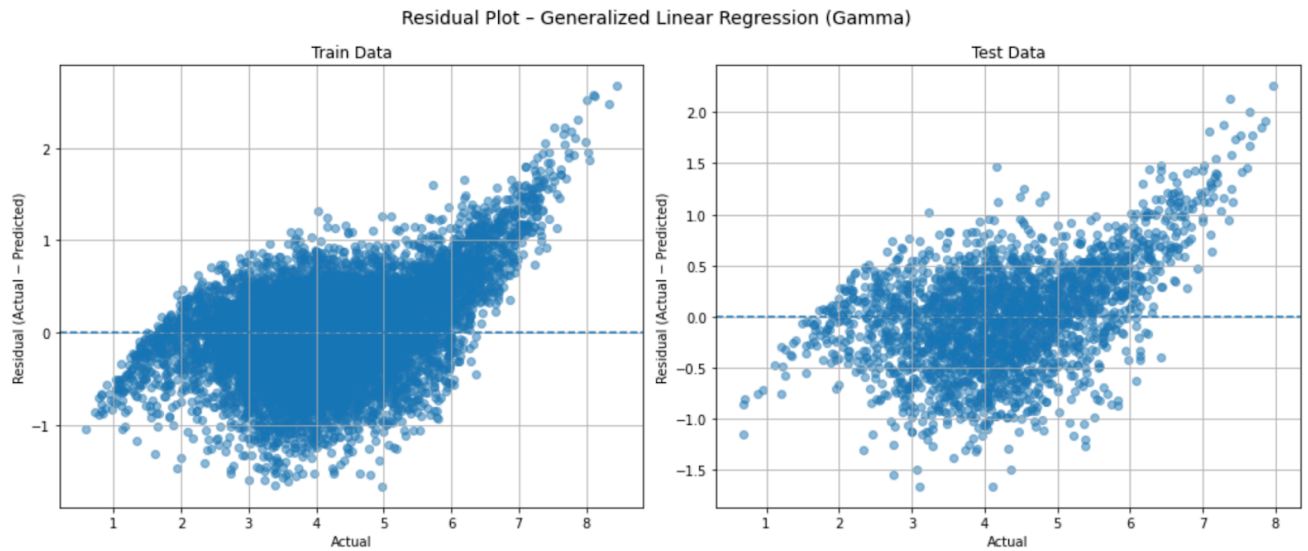
plot_residuals_vs_actual(
    glr_gaussian_model_predictions_train_data,
    glr_gaussian_model_predictions_test_data,
    "Generalized Linear Regression (Gaussian)"
)

```



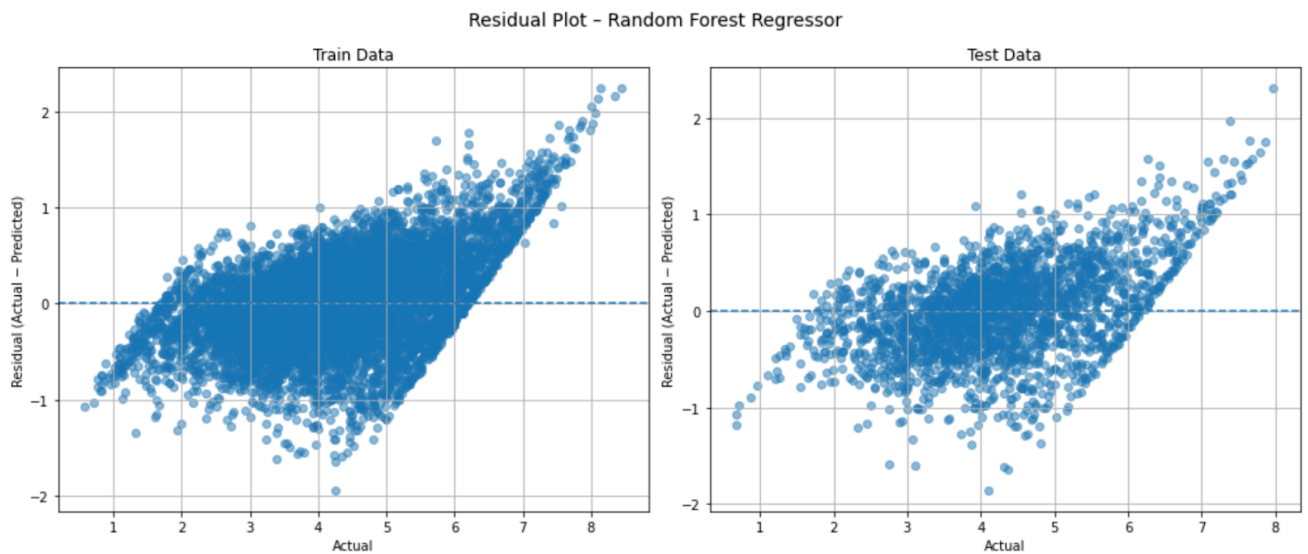
```
%spark.pyspark
%matplotlib inline

plot_residuals_vs_actual(
    glr_gamma_model_predictions_train_data,
    glr_gamma_model_predictions_test_data,
    "Generalized Linear Regression (Gamma)"
)
```



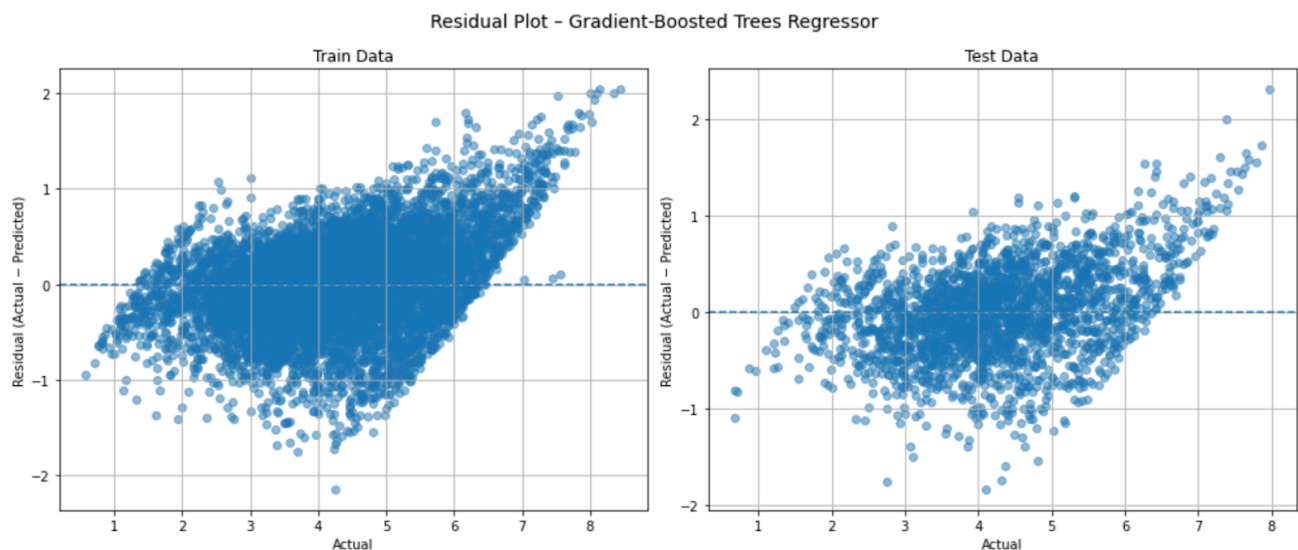
```
%spark.pyspark
%matplotlib inline

plot_residuals_vs_actual(
    rfr_model_predictions_train_data,
    rfr_model_predictions_test_data,
    "Random Forest Regressor"
)
```



```
%spark.pyspark
%matplotlib inline

plot_residuals_vs_actual(
    gbtr_model_predictions_train_data,
    gbtr_model_predictions_test_data,
    "Gradient-Boosted Trees Regressor"
)
```



17. Visualizing Residual Distributions

This step generates histograms of residuals for each model on both training and test datasets.

- The plots show the frequency distribution of prediction errors, helping to assess symmetry, bias, or outliers in model performance.
- A vertical line at zero indicates perfect predictions.

```
%spark.pyspark

def get_residuals(df):
    return (
        df.selectExpr("label - prediction as residual")
        .toPandas()
    )

def plot_residual_distributions(train_df, test_df, model_name, bins=50):
    train_res = get_residuals(train_df)
    test_res = get_residuals(test_df)

    fig, axes = plt.subplots(1, 2, figsize=(14, 6))

    # Train residuals
    axes[0].hist(train_res["residual"], bins=bins, alpha=0.7)
```

```

axes[0].axvline(0, linestyle="--")
axes[0].set_title("Train Residuals")
axes[0].set_xlabel("Residual (Actual - Predicted)")
axes[0].set_ylabel("Frequency")
axes[0].grid(True)

# Test residuals
axes[1].hist(test_res["residual"], bins=bins, alpha=0.7)
axes[1].axvline(0, linestyle="--")
axes[1].set_title("Test Residuals")
axes[1].set_xlabel("Residual (Actual - Predicted)")
axes[1].set_ylabel("Frequency")
axes[1].grid(True)

fig.suptitle(f"Residual Distributions - {model_name}", fontsize=14)
plt.tight_layout()
plt.show()

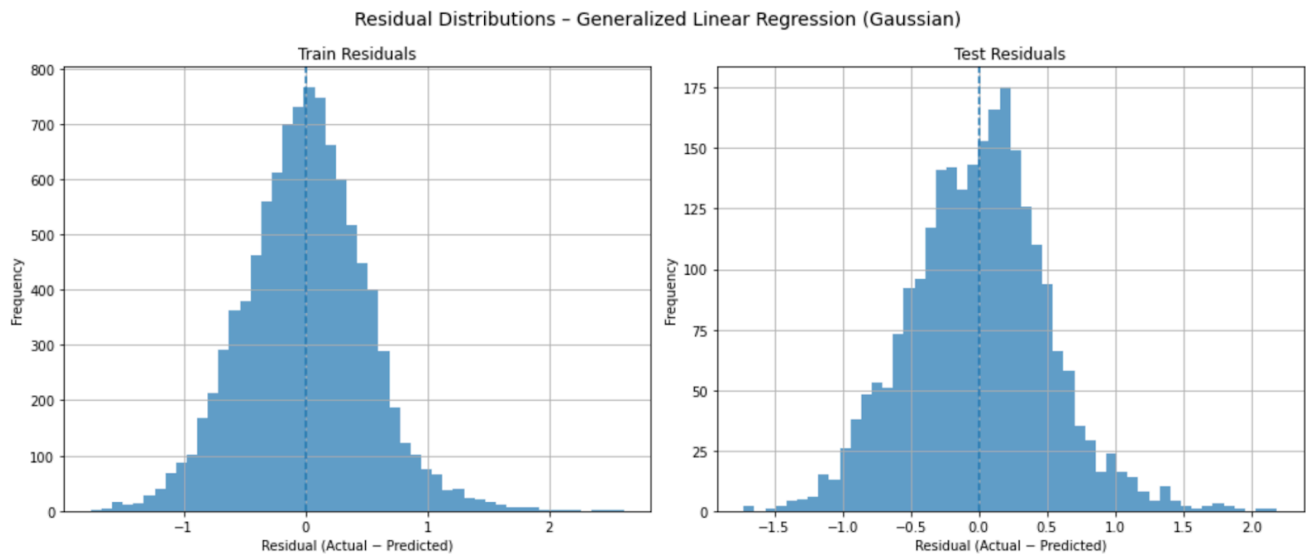
```

```

%spark.pyspark
%matplotlib inline

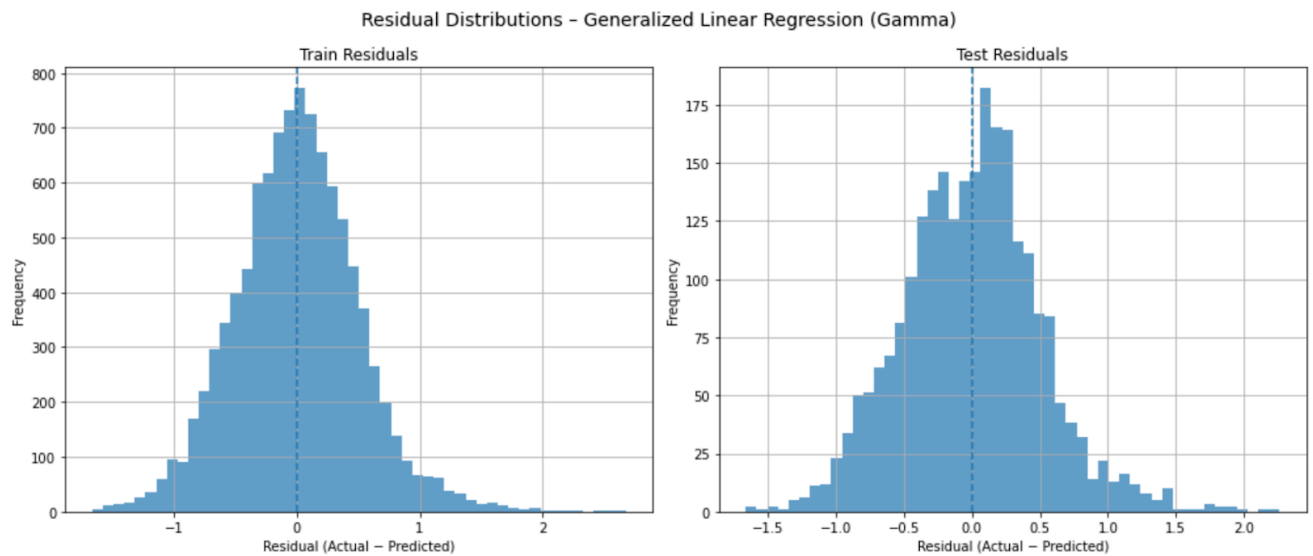
plot_residual_distributions(
    glr_gaussian_model_predictions_train_data,
    glr_gaussian_model_predictions_test_data,
    "Generalized Linear Regression (Gaussian)"
)

```



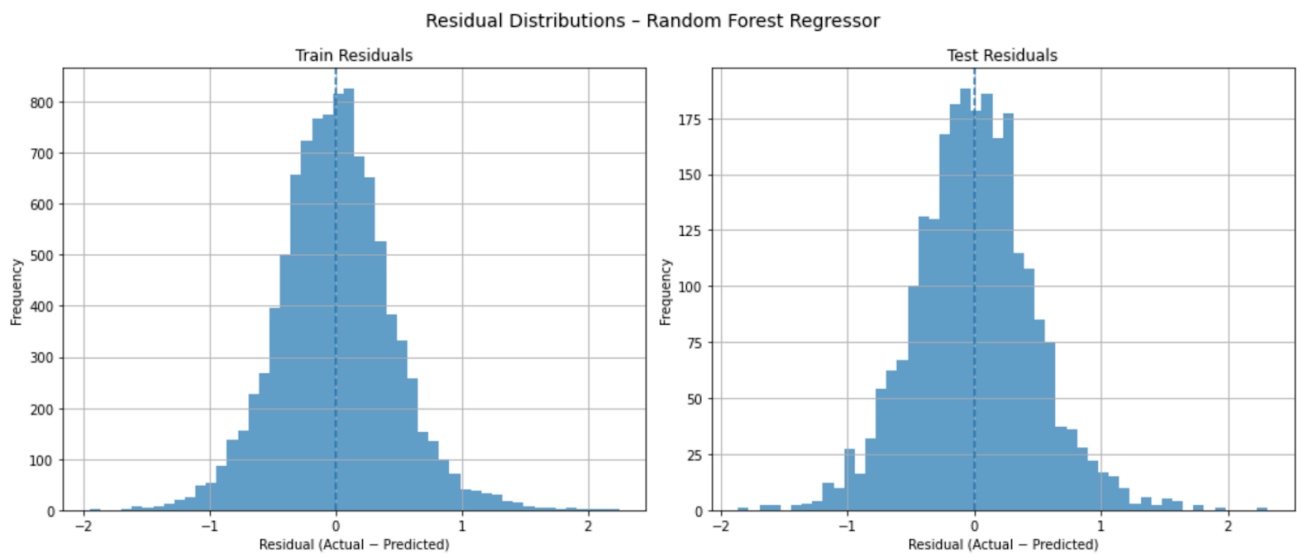
```
%spark.pyspark
%matplotlib inline

plot_residual_distributions(
    glr_gamma_model_predictions_train_data,
    glr_gamma_model_predictions_test_data,
    "Generalized Linear Regression (Gamma)"
)
```



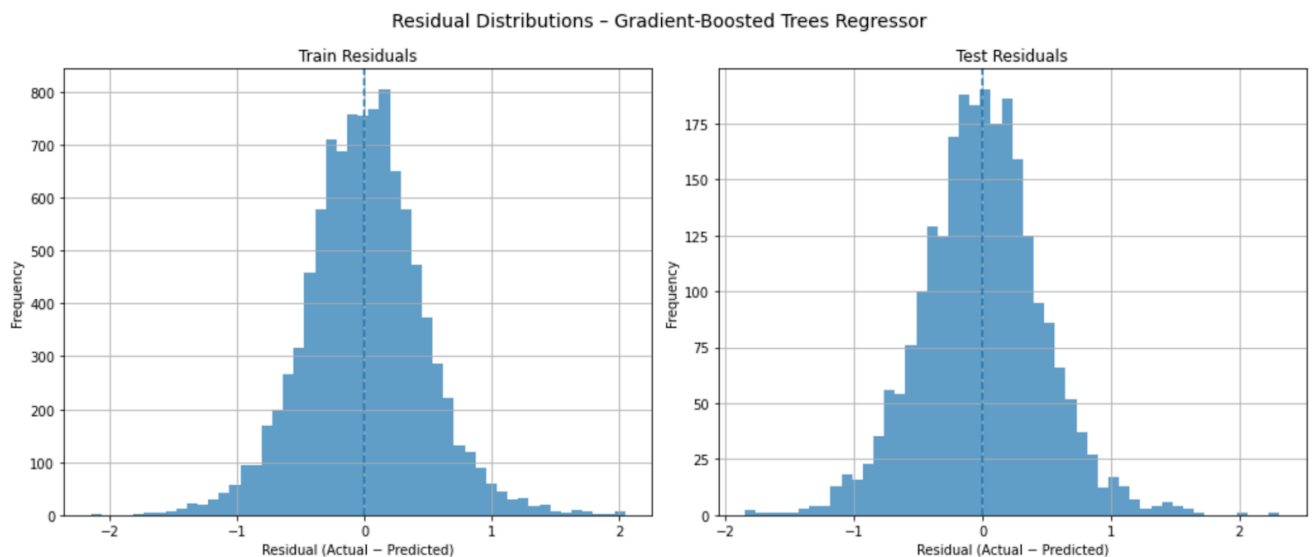
```
%spark.pyspark
%matplotlib inline

plot_residual_distributions(
    rfr_model_predictions_train_data,
    rfr_model_predictions_test_data,
    "Random Forest Regressor"
)
```



```
%spark.pyspark
%matplotlib inline

plot_residual_distributions(
    gbtr_model_predictions_train_data,
    gbtr_model_predictions_test_data,
    "Gradient-Boosted Trees Regressor"
)
```



18. Analyzing Feature Importances for Tree-Based Models

This step extracts and visualizes the importance of each input feature for the **Random Forest** and **Gradient-Boosted** Trees models.

- Feature importance scores indicate how much each feature contributes to the model's predictions.
- Bar plots are created for both models, ranking features from most to least influential.
- This analysis helps interpret the models and identify which weather variables (`precipitation_hours`, `sunshine_duration`, `wind_speed`) have the greatest impact on evapotranspiration.

```
%spark.pyspark

rf_importance = rfr_model.bestModel.featureImportances.toArray()
gbt_importance = gbtr_model.bestModel.featureImportances.toArray()

feature_names = assembler.getInputCols()

rf_imp_df = pd.DataFrame({
    "feature": feature_names,
    "importance": rf_importance
}).sort_values("importance", ascending=False)
```

```

gbt_imp_df = pd.DataFrame({
    "feature": feature_names,
    "importance": gbt_importance
}).sort_values("importance", ascending=False)

def plot_feature_importance(df, model_name):
    plt.figure(figsize=(10, 6))
    plt.barh(df["feature"], df["importance"])
    plt.gca().invert_yaxis()
    plt.xlabel("Importance Score")
    plt.title(f"Feature Importances - {model_name}")
    plt.grid(axis="x")
    plt.tight_layout()
    plt.show()

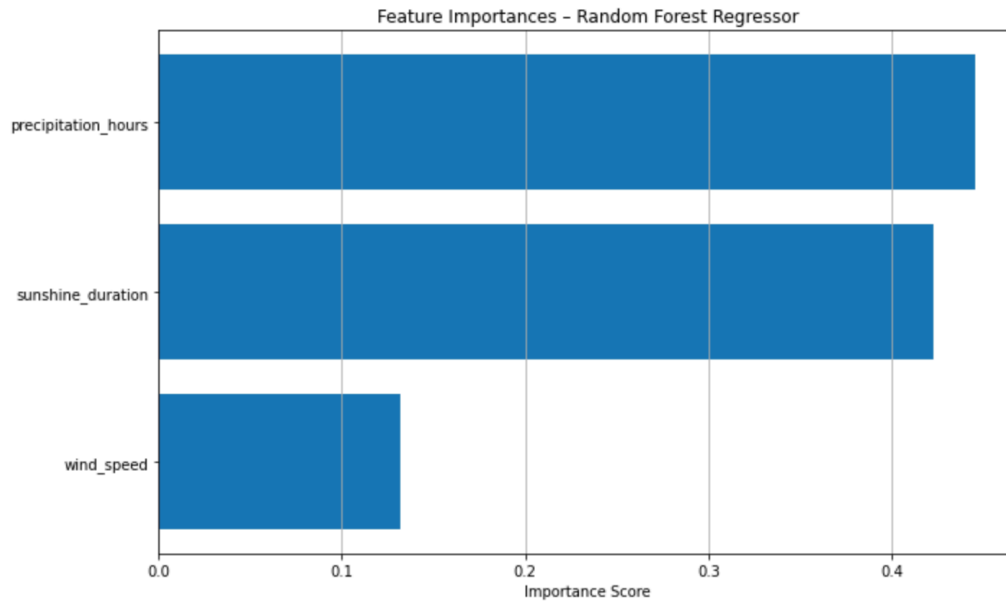
```

```

%spark.pyspark
%matplotlib inline

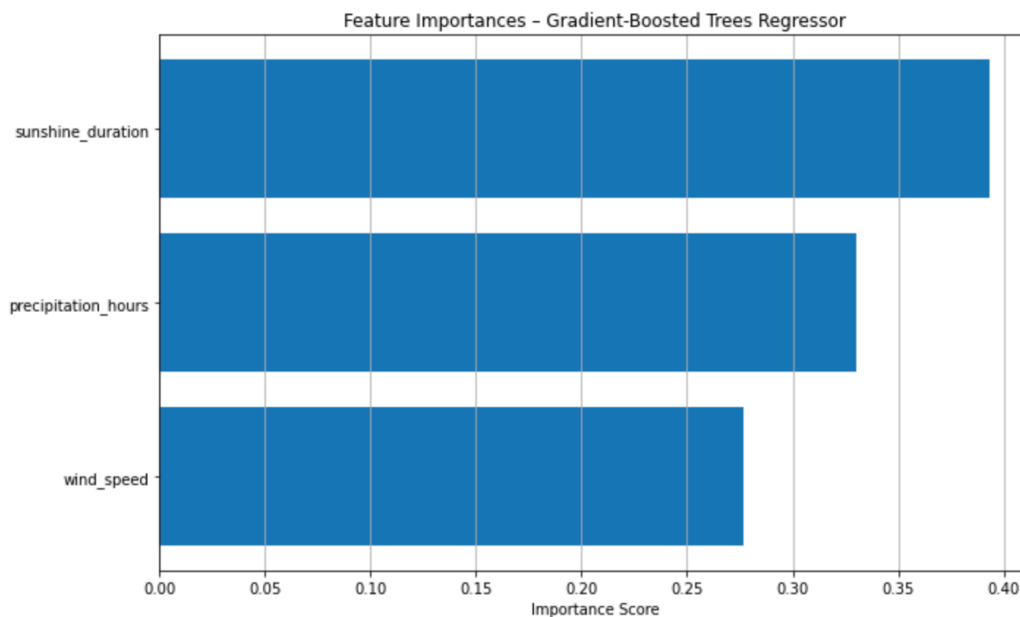
plot_feature_importance(
    rf_imp_df,
    "Random Forest Regressor"
)

```



```
%spark.pyspark
%matplotlib inline

plot_feature_importance(
    gbt_imp_df,
    "Gradient-Boosted Trees Regressor"
)
```



19. Determining Minimum and Maximum Values of Input Features

This step calculates the **minimum** and **maximum** values for each input feature (**precipitation_hours**, **sunshine_duration**, **wind_speed**) in the filtered May dataset.

- These statistics provide the observed ranges of the features, which can guide interpretation and help identify realistic input conditions for predicting lower evapotranspiration.
- The results are displayed in a concise table for reference in subsequent analysis.

```
%spark.pyspark

data_filtered_for_min_max = data_filtered_for_selected.select(
    pyspark_sql_functions_min("precipitation_hours").alias("min_precipitation_hours"),
    pyspark_sql_functions_max("precipitation_hours").alias("max_precipitation_hours"),
    pyspark_sql_functions_min("sunshine_duration").alias("min_sunshine_duration"),
    pyspark_sql_functions_max("sunshine_duration").alias("max_sunshine_duration"),
    pyspark_sql_functions_min("wind_speed").alias("min_wind_speed"),
    pyspark_sql_functions_max("wind_speed").alias("max_wind_speed")
)
```

```
%spark.pyspark
```

```
data_filtered_for_min_max.show(truncate=False)
```

min_precipitation_hours	max_precipitation_hours	min_sunshine_duration	max_sunshine_duration	min_wind_speed	max_wind_speed
0	24	0.0	142149.11	2.5	148.3

20. Generating Feature Scenarios for Prediction

This step creates a comprehensive grid of possible input feature combinations to explore how different values of `precipitation_hours`, `sunshine_duration`, and `wind_speed` affect evapotranspiration.

```
%spark.pyspark
```

```
min_precipitation_hours = 0
max_precipitation_hours = 24
samples_precipitation_hours = int((max_precipitation_hours - min_precipitation_hours) * 4)
+ 1

min_sunshine_duration = 0.0
max_sunshine_duration =
math.ceil(data_filtered_for_min_max.select("max_sunshine_duration").first()[0] / 3600) *
3600
samples_sunshine_duration = int((max_sunshine_duration - min_sunshine_duration) * 4 / 3600)
+ 1

min_wind_speed = 0
max_wind_speed = math.ceil(data_filtered_for_min_max.select("max_wind_speed").first()[0] /
10) * 10
samples_wind_speed = int((max_wind_speed - min_wind_speed)) + 1
```

```
%spark.pyspark
```

```
precipitation_hours_range = np.linspace(min_precipitation_hours, max_precipitation_hours,
samples_precipitation_hours)
sunshine_duration_range = np.linspace(min_sunshine_duration, max_sunshine_duration,
samples_sunshine_duration)
wind_speed_range = np.linspace(min_wind_speed, max_wind_speed, samples_wind_speed)
```

```
%spark.pyspark
```

```
scenarios = list(product(
    precipitation_hours_range,
    sunshine_duration_range,
    wind_speed_range
```

```

))

scenarios = [
    (float(p), float(s), float(w))
    for p, s, w in scenarios
]

scenario_df = spark.createDataFrame(
    scenarios,
    ["precipitation_hours", "sunshine_duration", "wind_speed"]
)

```

```

%spark.pyspark

scenario_df.count()

```

242403

21. Preparing Scenarios and Generating Model Predictions

```

%spark.pyspark

scenario_df_assembled = assembler.transform(scenario_df)
scenario_df_scaled = scaler_model.transform(scenario_df_assembled)

```

```

%spark.pyspark

scenario_df_scaled_predictions_glr_gaussian_model =
glr_gaussian_model.transform(scenario_df_scaled)
scenario_df_scaled_predictions_glr_gamma_model =
glr_gamma_model.transform(scenario_df_scaled)
scenario_df_scaled_predictions_rfr_model = rfr_model.transform(scenario_df_scaled)
scenario_df_scaled_predictions_gbtr_model = gbtr_model.transform(scenario_df_scaled)

```

```
%spark.pyspark
```

```
scenario_df_scaled_predictions_glr_gaussian_model.show(truncate=False)
```

precipitation_hours	sunshine_duration	wind_speed	features	scaled_features	prediction
10.0	10.0	10.0	I(3,[],[])	I[-1.1525516809243779,-3.7424631922613942,-2.317191502821753]	14.074030301733484
10.0	10.0	11.0	I[0.0,0.0,1.0]	I[-1.1525516809243779,-3.7424631922613942,-2.1875261626936164]	14.344455151896438
10.0	10.0	12.0	I[0.0,0.0,2.0]	I[-1.1525516809243779,-3.7424631922613942,-2.0578608225654795]	14.632830187543908
10.0	10.0	13.0	I[0.0,0.0,3.0]	I[-1.1525516809243779,-3.7424631922613942,-1.928195482437343]	14.940346901095079
10.0	10.0	14.0	I[0.0,0.0,4.0]	I[-1.1525516809243779,-3.7424631922613942,-1.7985301423092066]	15.268275873521521
10.0	10.0	15.0	I[0.0,0.0,5.0]	I[-1.1525516809243779,-3.7424631922613942,-1.66886480218107]	15.617972024065116
10.0	10.0	16.0	I[0.0,0.0,6.0]	I[-1.1525516809243779,-3.7424631922613942,-1.5391994620529335]	15.990880208420309
10.0	10.0	17.0	I[0.0,0.0,7.0]	I[-1.1525516809243779,-3.7424631922613942,-1.4095341219247968]	16.38854118851094
10.0	10.0	18.0	I[0.0,0.0,8.0]	I[-1.1525516809243779,-3.7424631922613942,-1.2798687817966603]	16.81259799852726
10.0	10.0	19.0	I[0.0,0.0,9.0]	I[-1.1525516809243779,-3.7424631922613942,-1.1502034416685236]	17.264802733525986
10.0	10.0	10.0	I[0.0,0.0,10.0]	I[-1.1525516809243779,-3.7424631922613942,-1.0205381015403872]	17.747023788642159
10.0	10.0	11.0	I[0.0,0.0,11.0]	I[-1.1525516809243779,-3.7424631922613942,-0.8908727614122505]	18.26125357882339
10.0	10.0	12.0	I[0.0,0.0,12.0]	I[-1.1525516809243779,-3.7424631922613942,-0.7612074212841139]	18.809616770982492
10.0	10.0	13.0	I[0.0,0.0,13.0]	I[-1.1525516809243779,-3.7424631922613942,-0.6315420811559773]	19.39437906258163

```
%spark.pyspark
```

```
scenario_df_scaled_predictions_glr_gamma_model.show(truncate=False)
```

precipitation_hours	sunshine_duration	wind_speed	features	scaled_features	prediction
10.0	10.0	10.0	I(3,[],[])	I[-1.1525516809243779,-3.7424631922613942,-2.317191502821753]	14.07970493638259
10.0	10.0	11.0	I[0.0,0.0,1.0]	I[-1.1525516809243779,-3.7424631922613942,-2.1875261626936164]	14.3418849432965265
10.0	10.0	12.0	I[0.0,0.0,2.0]	I[-1.1525516809243779,-3.7424631922613942,-2.0578608225654795]	14.620913804992188
10.0	10.0	13.0	I[0.0,0.0,3.0]	I[-1.1525516809243779,-3.7424631922613942,-1.928195482437343]	14.917874303909001
10.0	10.0	14.0	I[0.0,0.0,4.0]	I[-1.1525516809243779,-3.7424631922613942,-1.7985301423092066]	15.233918806907788
10.0	10.0	15.0	I[0.0,0.0,5.0]	I[-1.1525516809243779,-3.7424631922613942,-1.66886480218107]	15.570273737075557
10.0	10.0	16.0	I[0.0,0.0,6.0]	I[-1.1525516809243779,-3.7424631922613942,-1.5391994620529335]	15.928244332908382
10.0	10.0	17.0	I[0.0,0.0,7.0]	I[-1.1525516809243779,-3.7424631922613942,-1.4095341219247968]	16.309219713340569
10.0	10.0	18.0	I[0.0,0.0,8.0]	I[-1.1525516809243779,-3.7424631922613942,-1.2798687817966603]	16.714678268275155
10.0	10.0	19.0	I[0.0,0.0,9.0]	I[-1.1525516809243779,-3.7424631922613942,-1.1502034416685236]	17.146193395533894
10.0	10.0	10.0	I[0.0,0.0,10.0]	I[-1.1525516809243779,-3.7424631922613942,-1.0205381015403872]	17.605439606489213
10.0	10.0	11.0	I[0.0,0.0,11.0]	I[-1.1525516809243779,-3.7424631922613942,-0.8908727614122505]	18.094199024071237
10.0	10.0	12.0	I[0.0,0.0,12.0]	I[-1.1525516809243779,-3.7424631922613942,-0.7612074212841139]	18.614368298365726
10.0	10.0	13.0	I[0.0,0.0,13.0]	I[-1.1525516809243779,-3.7424631922613942,-0.6315420811559773]	19.167965966639091

```
%spark.pyspark
```

```
scenario_df_scaled_predictions_rfr_model.show(truncate=False)
```

precipitation_hours	sunshine_duration	wind_speed	features	scaled_features	prediction
10.0	10.0	10.0	I(3,[],[])	I[-1.1525516809243779,-3.7424631922613942,-2.317191502821753]	13.7288017158754725
10.0	10.0	11.0	I[0.0,0.0,1.0]	I[-1.1525516809243779,-3.7424631922613942,-2.1875261626936164]	14.199642278992558
10.0	10.0	12.0	I[0.0,0.0,2.0]	I[-1.1525516809243779,-3.7424631922613942,-2.0578608225654795]	14.681247525035116
10.0	10.0	13.0	I[0.0,0.0,3.0]	I[-1.1525516809243779,-3.7424631922613942,-1.928195482437343]	15.129503878292906
10.0	10.0	14.0	I[0.0,0.0,4.0]	I[-1.1525516809243779,-3.7424631922613942,-1.7985301423092066]	15.1498343809693266
10.0	10.0	15.0	I[0.0,0.0,5.0]	I[-1.1525516809243779,-3.7424631922613942,-1.66886480218107]	15.1498343809693266
10.0	10.0	16.0	I[0.0,0.0,6.0]	I[-1.1525516809243779,-3.7424631922613942,-1.5391994620529335]	15.1498343809693266
10.0	10.0	17.0	I[0.0,0.0,7.0]	I[-1.1525516809243779,-3.7424631922613942,-1.4095341219247968]	15.1498343809693266
10.0	10.0	18.0	I[0.0,0.0,8.0]	I[-1.1525516809243779,-3.7424631922613942,-1.2798687817966603]	15.1498343809693266
10.0	10.0	19.0	I[0.0,0.0,9.0]	I[-1.1525516809243779,-3.7424631922613942,-1.1502034416685236]	15.1498343809693266
10.0	10.0	10.0	I[0.0,0.0,10.0]	I[-1.1525516809243779,-3.7424631922613942,-1.0205381015403872]	15.1498343809693266
10.0	10.0	11.0	I[0.0,0.0,11.0]	I[-1.1525516809243779,-3.7424631922613942,-0.8908727614122505]	15.1498343809693266
10.0	10.0	12.0	I[0.0,0.0,12.0]	I[-1.1525516809243779,-3.7424631922613942,-0.7612074212841139]	15.1498343809693266
10.0	10.0	13.0	I[0.0,0.0,13.0]	I[-1.1525516809243779,-3.7424631922613942,-0.6315420811559773]	15.1498343809693266

```
%spark.pyspark
```

```
scenario_df_scaled_predictions_gbtr_model.show(truncate=False)
```

precipitation_hours	sunshine_duration	wind_speed	features	scaled_features	prediction
10.0	10.0	10.0	[(3, [], [])]	[-1.1525516809243779, -3.7424631922613942, -2.317191502821753]	13.791062859584747
10.0	10.0	11.0	[(0.0, 0.0, 1.0)]	[-1.1525516809243779, -3.7424631922613942, -2.1875261626936164]	14.087873858071027
10.0	10.0	12.0	[(0.0, 0.0, 2.0)]	[-1.1525516809243779, -3.7424631922613942, -2.0578608225654795]	14.78665861597522
10.0	10.0	13.0	[(0.0, 0.0, 3.0)]	[-1.1525516809243779, -3.7424631922613942, -1.928195482437343]	15.104831142373472
10.0	10.0	14.0	[(0.0, 0.0, 4.0)]	[-1.1525516809243779, -3.7424631922613942, -1.7985301423092066]	15.137895508109239
10.0	10.0	15.0	[(0.0, 0.0, 5.0)]	[-1.1525516809243779, -3.7424631922613942, -1.66886480218107]	15.137895508109239
10.0	10.0	16.0	[(0.0, 0.0, 6.0)]	[-1.1525516809243779, -3.7424631922613942, -1.5391994620529335]	15.137895508109239
10.0	10.0	17.0	[(0.0, 0.0, 7.0)]	[-1.1525516809243779, -3.7424631922613942, -1.4095341219247968]	15.137895508109239
10.0	10.0	18.0	[(0.0, 0.0, 8.0)]	[-1.1525516809243779, -3.7424631922613942, -1.2798687817966603]	15.137895508109239
10.0	10.0	19.0	[(0.0, 0.0, 9.0)]	[-1.1525516809243779, -3.7424631922613942, -1.1502034416685236]	15.137895508109239
10.0	10.0	10.0	[(0.0, 0.0, 10.0)]	[-1.1525516809243779, -3.7424631922613942, -1.0205381015403872]	15.137895508109239
10.0	10.0	11.0	[(0.0, 0.0, 11.0)]	[-1.1525516809243779, -3.7424631922613942, -0.8908727614122505]	15.137895508109239
10.0	10.0	12.0	[(0.0, 0.0, 12.0)]	[-1.1525516809243779, -3.7424631922613942, -0.7612074212841139]	15.137895508109239
10.0	10.0	13.0	[(0.0, 0.0, 13.0)]	[-1.1525516809243779, -3.7424631922613942, -0.6315420811559773]	15.137895508109239

22. Identifying Scenarios with Low Evapotranspiration

This step filters the scenario predictions to identify cases where the predicted evapotranspiration is **less than 1.5 mm**.

- Each model (GLR Gaussian, GLR Gamma, Random Forest, GBT) is used to generate these low-ET scenarios.
- The number of scenarios meeting this criterion is counted and displayed for each model.
- The resulting filtered DataFrames are also shown, providing specific feature combinations that are expected to result in lower evapotranspiration.

```
%spark.pyspark
```

```
low_et_predictions_glr_gaussian_model =  
scenario_df_scaled_predictions_glr_gaussian_model.filter(scenario_df_scaled_predictions_glr_gaussian_model.prediction < 1.5)  
low_et_predictions_glr_gamma_model =  
scenario_df_scaled_predictions_glr_gamma_model.filter(scenario_df_scaled_predictions_glr_gamma_model.prediction < 1.5)  
low_et_predictions_rfr_model =  
scenario_df_scaled_predictions_rfr_model.filter(scenario_df_scaled_predictions_rfr_model.prediction < 1.5)  
low_et_predictions_gbtr_model =  
scenario_df_scaled_predictions_gbtr_model.filter(scenario_df_scaled_predictions_gbtr_model.prediction < 1.5)
```

```
%spark.pyspark

print("Low ET predictions (< 1.5):")

print("GLR Gaussian model:", low_et_predictions_glr_gaussian_model.count())
print("GLR Gamma model  :", low_et_predictions_glr_gamma_model.count())
print("Random Forest    :", low_et_predictions_rfr_model.count())
print("Gradient Boosted  :", low_et_predictions_gbtr_model.count())
```

Low ET predictions (< 1.5):

GLR Gaussian model: 1264

GLR Gamma model : 1098

Random Forest : 0

Gradient Boosted : 0

```
%spark.pyspark

low_et_predictions_glr_gaussian_model.show()
```

	precipitation_hours	sunshine_duration	wind_speed	features	scaled_features	prediction
	7.25	0.0	0.0	[7.25,0.0,0.0]	[-0.1076855482402...	1.495325829127751
	7.5	0.0	0.0	[7.5,0.0,0.0]	[-0.0716556815959...	1.4445277509850025
	7.75	0.0	0.0	[7.75,0.0,0.0]	[-0.0356258149516...	1.3954553467340114
	7.75	0.0	1.0	[7.75,0.0,1.0]	[-0.0356258149516...	1.4880824935888275
	8.0	0.0	0.0	[8.0,0.0,0.0]	[4.04051692582621...	1.3480499930864658
	8.0	0.0	1.0	[8.0,0.0,1.0]	[4.04051692582621...	1.4375304805628268
	8.25	0.0	0.0	[8.25,0.0,0.0]	[0.03643391833686...	1.3022550582600658
	8.25	0.0	1.0	[8.25,0.0,1.0]	[0.03643391833686...	1.3886957822905384
	8.25	0.0	2.0	[8.25,0.0,2.0]	[0.03643391833686...	1.4808742446569214
	8.5	0.0	0.0	[8.5,0.0,0.0]	[0.07246378498114...	1.2580158343245893
	8.5	0.0	1.0	[8.5,0.0,1.0]	[0.07246378498114...	1.3415200594539647
	8.5	0.0	2.0	[8.5,0.0,2.0]	[0.07246378498114...	1.430567104811991
	8.75	0.0	0.0	[8.75,0.0,0.0]	[0.10849365162542...	1.2152794718462443
	8.75	0.0	1.0	[8.75,0.0,1.0]	[0.10849365162542...	1.295946954594298

```
%spark.pyspark
```

```
low_et_predictions_glr_gamma_model.show()
```

precipitation_hours	sunshine_duration	wind_speed	features	scaled_features	prediction
8.0	0.0	0.0	[8.0,0.0,0.0]	[4.04051692582621...	1.4845904086646384
8.25	0.0	0.0	[8.25,0.0,0.0]	[0.03643391833686...	1.4384249574196912
8.5	0.0	0.0	[8.5,0.0,0.0]	[0.07246378498114...	1.3936950865720112
8.5	0.0	1.0	[8.5,0.0,1.0]	[0.07246378498114...	1.483260139224413
8.75	0.0	0.0	[8.75,0.0,0.0]	[0.10849365162542...	1.350356154706396
8.75	0.0	1.0	[8.75,0.0,1.0]	[0.10849365162542...	1.4371360546006073
9.0	0.0	0.0	[9.0,0.0,0.0]	[0.14452351826970...	1.3083649085959717
9.0	0.0	1.0	[9.0,0.0,1.0]	[0.14452351826970...	1.3924462640201218
9.0	0.0	2.0	[9.0,0.0,2.0]	[0.14452351826970...	1.481931061774095
9.25	0.0	0.0	[9.25,0.0,0.0]	[0.18055338491398...	1.2676794400345004
9.25	0.0	1.0	[9.25,0.0,1.0]	[0.18055338491398...	1.3491461660687611
9.25	0.0	2.0	[9.25,0.0,2.0]	[0.18055338491398...	1.4358483067048085
9.5	0.0	0.0	[9.5,0.0,0.0]	[0.21658325155826...	1.2282591440110504
9.5	0.0	1.0	[9.5,0.0,1.0]	[0.21658325155826...	1.3071925462767688

```
%spark.pyspark
```

```
low_et_predictions_rfr_model.show()
```

precipitation_hours	sunshine_duration	wind_speed	features	scaled_features	prediction
---------------------	-------------------	------------	----------	-----------------	------------

```
%spark.pyspark
```

```
low_et_predictions_gbtr_model.show()
```

precipitation_hours	sunshine_duration	wind_speed	features	scaled_features	prediction
---------------------	-------------------	------------	----------	-----------------	------------

23. Calculating Mean Feature Values for Low Evapotranspiration Scenarios

This step computes the average input feature values (`precipitation_hours`, `sunshine_duration`, `wind_speed`) for scenarios predicted to have evapotranspiration below 1.5 mm.

- The calculations are performed separately for the **GLR Gaussian** and **GLR Gamma** models.
- These mean values summarize the typical weather conditions expected to result in **lower evapotranspiration for May 2026**, providing actionable insights from the predictive models.

```
%spark.pyspark

mean_values_glr_gaussian_model = low_et_predictions_glr_gaussian_model.select(
    mean('precipitation_hours').alias('mean_precipitation_hours'),
    mean('sunshine_duration').alias('mean_sunshine_duration'),
    mean('wind_speed').alias('mean_wind_speed')
).collect()[0]

print("Predicted mean conditions for May 2026 with evapotranspiration < 1.5 mm:
GeneralizedLinearRegression (gaussian) Model")
print(f"Precipitation hours:
{mean_values_glr_gaussian_model['mean_precipitation_hours']:.2f}")
print(f"Sunshine duration: {mean_values_glr_gaussian_model['mean_sunshine_duration']:.2f}")
print(f"Wind speed: {mean_values_glr_gaussian_model['mean_wind_speed']:.2f}")
```

```
Predicted mean conditions for May 2026 with evapotranspiration < 1.5 mm: GeneralizedLinearRegression (gaussian) Model
Precipitation hours: 18.41
Sunshine duration: 0.00
Wind speed: 11.80
```

```
%spark.pyspark

mean_values_glr_gamma_model = low_et_predictions_glr_gamma_model.select(
    mean('precipitation_hours').alias('mean_precipitation_hours'),
    mean('sunshine_duration').alias('mean_sunshine_duration'),
    mean('wind_speed').alias('mean_wind_speed')
).collect()[0]

print("Predicted mean conditions for May 2026 with evapotranspiration < 1.5 mm:
GeneralizedLinearRegression (gamma) Model")
print(f"Precipitation hours:
{mean_values_glr_gamma_model['mean_precipitation_hours']:.2f}")
print(f"Sunshine duration: {mean_values_glr_gamma_model['mean_sunshine_duration']:.2f}")
print(f"Wind speed: {mean_values_glr_gamma_model['mean_wind_speed']:.2f}")
```

```
Predicted mean conditions for May 2026 with evapotranspiration < 1.5 mm: GeneralizedLinearRegression (gamma) Model
Precipitation hours: 18.65
Sunshine duration: 0.00
Wind speed: 10.65
```

24. Verifying Predicted Evapotranspiration for Mean Low-ET Conditions

This step validates that the mean weather conditions calculated from the low-ET scenarios actually lead to the desired low evapotranspiration according to the trained models.

```
%spark.pyspark

mean_input_glr_gaussian_df = spark.createDataFrame([
    Row(
        precipitation_hours=float(mean_values_glr_gaussian_model['mean_precipitation_hours']),
        sunshine_duration=float(mean_values_glr_gaussian_model['mean_sunshine_duration']),
        wind_speed=float(mean_values_glr_gaussian_model['mean_wind_speed'])
    )
])

mean_input_glr_gaussian_assembled_df = assembler.transform(mean_input_glr_gaussian_df)
mean_input_glr_gaussian_scaled_df =
scaler_model.transform(mean_input_glr_gaussian_assembled_df)

mean_input_glr_gaussian_prediction_df =
glr_gaussian_model.transform(mean_input_glr_gaussian_scaled_df)

mean_input_glr_gaussian_prediction_value =
mean_input_glr_gaussian_prediction_df.collect()[0]['prediction']

print(f"Predicted ET0 (Gaussian GLR): {mean_input_glr_gaussian_prediction_value:.3f} mm")

if mean_input_glr_gaussian_prediction_value < 1.5:
    print("✅ Mean conditions satisfy ET0 < 1.5 mm")
else:
    print("❌ Mean conditions do NOT satisfy ET0 < 1.5 mm")
```

Predicted ET₀ (Gaussian GLR): 0.682 mm

✅ Mean conditions satisfy ET₀ < 1.5 mm

```

%spark.pyspark

mean_input_glr_gamma_df = spark.createDataFrame([
    Row(
        precipitation_hours=float(mean_values_glr_gamma_model['mean_precipitation_hours']),
        sunshine_duration=float(mean_values_glr_gamma_model['mean_sunshine_duration']),
        wind_speed=float(mean_values_glr_gamma_model['mean_wind_speed'])
    )
])

mean_input_glr_gamma_assembled_df = assembler.transform(mean_input_glr_gamma_df)
mean_input_glr_gamma_scaled_df = scaler_model.transform(mean_input_glr_gamma_assembled_df)

mean_input_glr_gamma_prediction_df =
glr_gamma_model.transform(mean_input_glr_gamma_scaled_df)

mean_input_glr_gamma_prediction_value =
mean_input_glr_gamma_prediction_df.collect()[0]['prediction']

print(f"Predicted ET0 (Gamma GLR): {mean_input_glr_gamma_prediction_value:.3f} mm")

if mean_input_glr_gamma_prediction_value < 1.5:
    print("✅ Mean conditions satisfy ET0 < 1.5 mm")
else:
    print("❌ Mean conditions do NOT satisfy ET0 < 1.5 mm")

```

Predicted ET₀ (Gamma GLR): 0.750 mm

✅ Mean conditions satisfy ET₀ < 1.5 mm

- Task 4 -

Dashboard Screenshots

